

Documento Requisiti — Ruffino Flow (PRD)

Stato: Documento vivente, riallineato allo stato corrente dell'applicazione (19/08/2026). **Versione:** 4.9 - Tars migrato a OpenAI Responses API: function calling, modelli separati per lavoro umano e automatico, prompt caching misurabile e limite strumenti a chiusura garantita. Base v4.8 - Smistamento Tars affidabile e osservabile con coda drenata per lotti, recupero dopo riavvio e stato operativo visibile. **Riferimento implementativo:** repository `infissi-ops-app`. Il presente PRD descrive il comportamento atteso del software così come è implementato; ogni divergenza riscontrata nel codice va trattata come bug.

0. Convenzioni del documento

- **MUST / DEVE** — requisito obbligatorio.
 - **SHOULD / DOVREBBE** — requisito fortemente consigliato.
 - **MAY / PUÒ** — opzione facoltativa.
 - Gli identificatori `in_questo_stile` sono valori interni (enum, chiavi di database). Le etichette UI in italiano sono indicate fra virgolette.
 - I numeri di sezione sono stabili; le sotto-sezioni possono crescere.
-

1. Visione del prodotto

Ruffino Flow è lo strumento operativo centrale di **Ruffino Immobiliare S.R.L.** Collega ufficio, laboratorio di produzione e cantiere, accompagnando ogni cliente dalla prima richiesta fino alla garanzia post-vendita. Non è un semplice database: è un assistente proattivo che ricorda le scadenze, blocca i passaggi di stato senza i documenti richiesti, unifica le comunicazioni e affianca gli operatori con proposte Tars sempre verificabili.

Pilastrì:

1. **Una commessa = un percorso tracciato.** Stato, documenti, interventi, anomalie e firme convivono in un unico fascicolo.
 2. **Niente dato perso.** Le commesse rifiutate dal cliente vengono **archivate**, non cancellate; in qualsiasi momento si possono ripristinare con stato e file invariati.
 3. **Sicurezza by default.** Autenticazione obbligatoria su ogni endpoint; password hashate; gate documentale lato server.
 4. **Operatività prima dell'eleganza.** Pagine come Calendario e Board mostrano in faccia all'utente le informazioni che gli servono (nome, cognome, indirizzo lavoro, telefono cliccabile).
-

2. Architettura tecnica (sintesi)

- **Frontend.** React 19 + Vite 7 + Wouter (routing) + tRPC 11 (client) + React Query (caching) + Tailwind 4 + shadcn/ui + lucide-react + sonner (toast) + jsPDF/jspdf-autotable. Le pagine sono caricate per rotta con React.lazy; i vendor sono separati per React, UI, dati e grafici.

- **Backend.** Node + Express + tRPC 11. Persistenza prevalente in `kv_store` (Postgres JSONB) tramite `persistedStore`, con save debounced, retry su errori transienti e recovery in background. Le Comunicazioni usano una tabella PostgreSQL dedicata.
 - **Autenticazione.** Locale via email/password con JWT firmato (jose, HS256, TTL 7 giorni) + cookie httpOnly. Sessione server-side cacheata in memoria con eviction periodica.
 - **Sicurezza.** Tutti gli endpoint business sono `protectedProcedure` (utente loggato obbligatorio); le mutazioni su `utenti` e l'intero router `backup / fattureInCloud` sono `adminProcedure` (ruolo direzione). Header `X-Content-Type-Options`, `X-Frame-Options=SAMEORIGIN`, `Referrer-Policy`, `HSTS` in produzione. Upload con allowlist mimeType + validazione reale del payload base64. CSRF same-origin check su `/api/trpc.trust proxy` abilitato (deploy dietro Railway).
 - **Scheduler interni.** Backup notturno Google Drive (00:00 Europe/Rome, `setTimeout` ri-armato), sync Fatture in Cloud (ogni 6 h quando abilitato), audit processi Tars (controllo ogni 6 h, massimo un run per sede ogni circa 24 h).
 - **PDF.** jsPDF + jspdf-autotable sia client-side (preventivatori, scheda cliente) sia server-side (scheda cliente nel backup).
 - **Storage file.** Driver `local` o S3-compatible/R2. I record conservano `storageKey` + checksum SHA-256; `dataBase64` resta supportato per i record legacy e come fallback in scrittura. Cap per-file 10 MB.
 - **Agente AI.** Tars usa OpenAI Responses API con function calling, strumenti read-only e proposte persisterite. Ogni modifica richiede approvazione umana e passa dalle mutation applicative (§50).
-

3. Autenticazione e sicurezza

3.1 Login

- Schermata `LoginPage` con email + password.
- Login solo per utenti `attivo === true`.
- Confronto password tramite `verifyPassword` (scrypt) — accetta anche password legacy in chiaro durante una migrazione trasparente.
- **Rate limit per email:** dopo 5 tentativi falliti in 15 minuti l'account è bloccato fino allo scadere della finestra. Login riuscito → reset del contatore.
- Messaggio di errore generico ("Email o password non validi") per non rivelare se l'utente esiste.

3.2 Sessione

- JWT firmato HS256, claim: `sub` (id utente), `email`, `name`, `role`, `ruolo`, `ruoli`. Esp. 7 giorni.
- Cookie `httpOnly`, `secure` su HTTPS, `sameSite none` su https / `lax` su http locale.
- Cache server-side `Map<token, { user, expMs }>` con eviction lazy e sweep orario (`setInterval(...).unref()`).
- **Logout** cancella sia il cookie sia l'entry della cache server.

3.3 Hashing password

- Algoritmo: **scrypt** (modulo `crypto` di Node, nessuna dipendenza esterna).
- Formato di archiviazione: `scrypt$<saltHex>$<hashHex>` (salt 16 byte, key 64 byte).
- `verifyPassword` è constant-time tramite `timingSafeEqual`.
- Le password create/aggiornate dalla UI sono sempre hashate.

- Le password nuove devono avere almeno 12 caratteri.
- Le password legacy in chiaro residue nel DB vengono **automaticamente upgrate a hash al primo boot successivo** all'aggiornamento (vedi `utenti.onLoad`).
- Su store utenti vuoto viene creato un solo amministratore da `BOOTSTRAP_ADMIN_*`; in produzione `B00-TSTRAP_ADMIN_PASSWORD` è obbligatoria. Non esistono più password seed fisse nel codice corrente.

3.4 Segreto JWT

- In **produzione** la variabile d'ambiente `JWT_SECRET` è OBBLIGATORIA: in sua assenza il server fa throw all'avvio.
- In dev/test, in mancanza di `JWT_SECRET`, viene usato un fallback hard-coded; **non valido per produzione**.

3.5 Header di sicurezza

Su ogni risposta HTTP:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: SAMEORIGIN`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `X-DNS-Prefetch-Control: off`
- `X-Permitted-Cross-Domain-Policies: none`
- In produzione: `Strict-Transport-Security: max-age=15552000; includeSubDomains`
- CSP **non** impostata di default (richiede tuning con Vite + Maps proxy + blob preview) — tracciata come follow-up.

3.6 Esposizione API

- Endpoint tRPC montati su `/api/trpc`.
- Tutti i router business utilizzano `protectedProcedure`. Le sole eccezioni pubbliche sono: `auth.me`, `auth.login`, `auth.logout`, `system.health`.
- `utenti.create`, `utenti.update`, `utenti.delete` richiedono `adminProcedure` (= ruolo `direzione`).

3.7 Protezioni aggiuntive (v4)

- **Hash versionato.** Formato `script$<N>$<saltHex>$<hashHex>` con `N=32768`; verifica retro-compatibile con il legacy `script$salt$hash` (`N=16384`) e con password in chiaro (upgrade automatico al boot).
- **CSRF.** Su `/api/trpc` le richieste con header `Origin` diverso dall'`Host` vengono rifiutate (403). Richieste server-to-server senza `Origin` ammesse.
- **SSRF guard (calendari esterni).** Gli URL iCal inseriti dagli operatori sono validati (solo https; vietati localhost/ .local/ .internal/ IP privati RFC1918/link-local/IPv6 raw) sia all'inserimento sia a ogni fetch.
- **Segreti mascherati in UI.** Gli indirizzi iCal privati (bearer secret) e i token Fatture in Cloud sono ritornati mascherati dalle list/status API.
- **Isolamento sede.** Ogni query/mutation è vincolata alla sede attiva (`ctx.sedeId`); guardie `assertSedeScope` su tutte le entità (vedi §34).

3.8 Operatività residua a carico del titolare (non risolvibile via codice)

- Rotazione manuale delle vecchie password seed eventualmente ancora attive e revoca dei token GitHub non riconosciuti.
 - Decisione e finestra coordinata per pulire lo storico Git (BFG / `git filter-repo` + `force-push`). Il codice corrente non contiene più password seed fisse, ma la cronologia resta immutata.
-

4. Ruoli, permessi e gating

4.1 Set di ruoli

- `direzione` — ammessi accesso completo + gestione utenti + sezioni gated (Squadre, Garanzie, Fornitori, Produzione, Utenti, Integrazioni avanzate).
- `amministrazione` — focus su fatturazione, finiture, saldo.
- `commerciale` — gestione clienti e commesse commerciali.
- `tecnico_rilievi` — rilievi e misure.
- `squadra_posa` — esecuzione in cantiere.
- `post_vendita` — ticket, reclami, rifacimenti.
- `ordini` — ordini fornitori.

Ogni utente ha `ruoli: string[]` (1–3 valori). Il campo legacy `ruolo` continua a contenere il ruolo primario per retro-compatibilità.

4.2 Mapping `role` legacy

- `role = "admin"` quando in `ruoli` è presente `direzione`. Altrimenti `user`.

4.3 Gating

- **Lato server.** `protectedProcedure` su tutto il business; `adminProcedure` su mutazioni `utenti` e `system.notifyOwner`.
 - **Lato client.** Componente `RequireDirezione` su rotte `Garanzie`, `Squadre`, `Fornitori`, `Produzione`, `Utenti`. Le voci di sidebar corrispondenti sono filtrate con il flag `direzioneOnly`.
-

5. Anagrafica Cliente

5.1 Tipi cliente

`privato` | `azienda` | `condominio` | `ente_pubblico`. Icone in UI: `User`, `Building2`, `Home`, `Landmark`.

5.2 Campi del Cliente

Anagrafica:

- **Tipo** (vedi 5.1; default `privato`) — è il **primo campo del form**: la sua scelta cambia i campi successivi.

- Se `tipo === "privato"`: **Cognome** e **Nome** (entrambi obbligatori).
- Se `tipo ∈ {azienda, condominio, ente_pubblico}`: un solo campo **Ragione sociale** (obbligatorio), archiviato nel campo `cognome`; `nome` viene valorizzato a spazio singolo. Stessa convenzione usata dalla sincronizzazione Fatture in Cloud (§40) e dalla migrazione 2026 (§43), così le denominazioni restano indivise.
- **Codice fiscale, partita IVA.**

Doppio indirizzo:

- **Indirizzo di residenza** (`indirizzo`, `citta`, `cap`) — usato dall'amministrazione per la fatturazione. Per i clienti non privati l'etichetta diventa «**Sede legale (fatturazione)**» ovunque: form di creazione/modifica, badge nella scheda cliente e scheda PDF (§42).
- **Indirizzo dei lavori** (`indirizzoLavoro`, `cittaLavoro`, `capLavoro`) — usato dalle commesse per cantiere, calendario, mappe.
- In fase di creazione/modifica il form propone uno **switch "stesso della residenza"** che copia automaticamente i campi residenza → lavoro al salvataggio.
- La scheda cliente espone entrambi gli indirizzi con badge distintivo "Residenza" / "Lavoro".

Contatti: **telefono, email.**

Dati fiscali e amministrativi:

- **Detrazione** (switch). Quando attivata RICHIEDE `tipoDetrazione`.
- **Tipo detrazione:** `ecobonus` | `ristrutturazione` (obbligatorio se `detrazione === true`).
- **Finanziamento** (switch).
- **Pratica edilizia:** `nessuna` | `cil` | `cila` | `scia`.

Referenti (lista multipla):

- Campi per referente: `nome`, `ruolo` (`cliente_finale` | `architetto` | `direttore_lavori` | `amministratore` | `altro`), `telefono`, `email`.

Operativi:

- **Note libere.**
- **Assegnato a** (utente). Default: utente corrente; eredita su nuove commesse linkate al cliente.

5.3 Comportamenti del Cliente

- Lista clienti con ricerca testuale, filtro per tipo, filtro "solo le mie".
 - Scheda cliente con tabs: **Commesse**, **Interventi**, **Ticket**, **Garanzie** — ognuno ha un pulsante **+** per creare l'entità collegata.
 - **Cascade su update.** Modificando nome/cognome o un campo di contatto/indirizzo del cliente, il server propaga il valore a tutte le commesse linkate (solo dove il valore della commessa coincideva con il valore precedente del cliente, così le override manuali non vengono sovrascritte).
 - Eliminazione cliente disponibile (con conferma) ma SCONSIGLIATA in presenza di commesse linkate.
 - **Scheda PDF** stampabile dall'header (vedi §42) e bottone **WhatsApp** accanto al telefono (vedi §41).
 - Ogni cliente appartiene a una sede (`sedeId`, vedi §34).
-

6. Commesse

6.1 Identificazione

- **Codice automatico** `COM-{ANNO}-{N}` con N progressivo a 3 cifre (es. `COM-2026-001`).
- Lo ZeroPadding è coerente per anno; al cambio anno il contatore può ripartire da 1.

6.2 Campi

- **clientId** (FK), **cliente** (display name denormalizzato, sincronizzato da `cliente.update`).
- **indirizzo, citta, telefono, email** — inizialmente derivati dall' `indirizzoLavoro` del cliente; sovrascrivibili a livello di commessa.
- **stato** — vedi 7.1.
- **priorita**: `bassa` | `media` | `alta` | `urgente`.
- **squadraId** (FK opzionale).
- **dataApertura** (auto).
- **consegnaIndicativa**: enum `"30"` | `"60"` | `"90"` (giorni dal preventivo).
- **dataConsegnaIndicativa**: data libera ISO. **Mutuamente esclusiva** con `consegnaIndicativa`: salvando una delle due l'altra viene azzerata.
- **dataConsegnaConfermata**: data definitiva impostata al passaggio in `produzione`.
- **dataChiusura**: impostata automaticamente al passaggio in `archiviata` (chiusura del flusso).
- **note** libere.
- **importoTotale**: totale pattuito (€, opzionale). **importoIncassato**: derivato — somma del registro `pagamenti` (vedi §37); non modificabile direttamente dalla UI.
- **pagamenti**: registro acconti embedded (vedi §37). Escluso da `commesse.list` come `prodotti`.
- **sedId**: sede proprietaria (vedi §34).
- **prodotti**: array di prodotti della commessa (nome, tipologia/materiale, quantità, dimensioni, note) — *di cosa si tratta*, vedi §44. NON viene incluso nella risposta di `commesse.list` per alleggerire i payload; viene ritornato da `commesse.byId`. La lista riceve invece la sintesi `prodottiSintesi`: `[{nome, quantita}]`.
- **costi**: registro dei costi fornitore embedded, base del calcolo del margine (vedi §45).
- **costoPosaStimato**: stima manuale del costo di posa (€, opzionale). Scrivibile solo da direzione o amministrazione.
- **squadraId**: squadra di posa assegnata alla commessa (vedi §46).
- **dataApertura**: data di creazione in formato `YYYY-MM-DD`, mostrata come "Creata il" nella scheda e in colonna nella lista.
- **assegnatoA** (FK utente). Modificabile dalla scheda commessa.
- **createdBy** (FK utente).
- **createdAt, updatedAt**.
- **archivedAt**: timestamp ISO; `null` finché la commessa non è in soft-archive (vedi §24).

6.3 Consegna indicativa — selezione

La UI offre quattro opzioni nel select **Consegna indicativa**:

1. `+30 giorni`
2. `+60 giorni` (*default*)
3. `+90 giorni`

4. **Data da calendario...** → mostra un date picker, popola `dataConsegnaIndicativa` e svuota `consegnaIndicativa`.

L'header della commessa mostra in ordine di priorità:

1. `dataConsegnaConfermata` (etichetta "Data consegna prevista").
2. `dataConsegnaIndicativa` (etichetta "Consegna indicativa", formato `gg/mm/aaaa`).
3. `consegnaIndicativa` (etichetta "Consegna indicativa: +N giorni").

6.4 Trigger Produzione

Quando una commessa entra nello stato `produzione`:

1. Sulla scheda compare un banner ambra "Commessa in produzione" con il pulsante "Aggiorna data consegna".
2. Al click si apre un dialog con date picker per la **Data di Consegna Prevista** definitiva.
3. Il salvataggio popola `dataConsegnaConfermata` (visibile in header, card Kanban, lista in Dashboard).
4. Finché `dataConsegnaConfermata` è vuota, la card del Kanban resta evidenziata con anello ambra e contatore "Consegne da confermare" nel KPI bar.

6.5 Modifica

- Dialog "**Modifica commessa**" consente di modificare contemporaneamente:
 - Anagrafica cliente collegata (nome, cognome, CF, P.IVA, CAP, telefono, email, indirizzo, città) — sincronizzata via `clienti.update` con cascade.
 - Priorità.
 - **Utente assegnato** (SearchSelect sugli utenti, opzione "Non assegnata").
 - Consegna indicativa (preset o data libera).
 - Note.
- Nell'header sono visibili pill: indirizzo, telefono, email, **Assegnata a: Nome** (lookup utente).

6.6 Eliminazione vs Archiviazione

- **Archivia** — operazione consigliata se il cliente non procede. Non distrugge nulla (vedi §24).
- **Elimina** — operazione distruttiva. Conferma esplicita. Dovrebbe essere usata solo per errori di inserimento.

6.7 Lista commesse

- Filtri: search testuale (codice, cliente, città), stato, clienteld, assegnatoA, scope `archived = exclude | only | all` (default `exclude`).
 - Ordinata per `createdAt desc`.
 - Risposta **non include** `prodotti` né `pagamenti` (ottimizzazione bandwidth/render). Include però `prodottiSintesi` (nome + quantità per riga) e `nPagamenti` (conteggio degli acconti), che alimentano rispettivamente la colonna Prodotti e la proposta della rata successiva nella pagina Pagamenti.
-

7. State machine delle commesse

7.1 Stati

Ordine canonico (STATI_COMMESSA):

1. preventivo
2. misure_esecutive
3. aggiornamento_contratto
4. fatture_pagamento
5. da_ordinare
6. produzione
7. ordini_ultimazione *(richiesta secondo acconto)*
8. attesa_posa
9. finiture_saldo
10. interventi_regolazioni
11. archiviata *(chiusura del flusso — distinta dal soft-archive di §24)*

7.2 Transizioni valide

- Transizioni **in avanti** consentite di **una posizione** (es. da_ordinare → produzione).
- Transizioni **all'indietro** consentite di **una posizione**.
- Salti multipli **vietati** lato server (validateTransizione).
- Stato archiviata raggiungibile solo dal predecessore interventi_regolazioni.

7.3 Soft-archive (orthogonal)

- archivedAt è ortogonale a stato . Una commessa può essere soft-archiviata in qualsiasi stato (vedi §24).
- Il soft-archive **non** modifica lo stato corrente.

7.4 Avanzamento via UI

- **Scheda commessa**: pulsante "Avanza" che propone lo stato successivo. Disabilitato se manca un documento richiesto (vedi §9), eccetto bypass con "Procedi comunque".
- **Board Kanban**: ogni card ha frecce **Indietro** / **Avanza**. Le frecce attraversano sempre la stessa state machine (controlli server identici).

7.5 Stati e gate documentale (vedi anche §9)

- preventivo → necessita preventivo o contratto .
- misure_esecutive → misure .
- aggiornamento_contratto → contratto .
- fatture_pagamento → fattura .
- da_ordinare → ordine o conferma_ordine .
- produzione → nessun documento richiesto (gated da dataConsegnaConfermata).
- ordini_ultimazione → saldo o fattura .
- attesa_posa → ddt_consegna .
- finiture_saldo → ddt_posa .

- `interventi_regolazioni` → `ddt_finale`.
 - `archiviata` → nessun documento richiesto.
-

8. Documenti commessa (Preventivi/Contratti)

8.1 Tipi documento

`preventivo`, `contratto`, `misure`, `fattura`, `ordine`, `conferma_ordine`, `ddt_consegna`, `ddt_posa`, `ddt_finale`, `saldo`, `foto`, `altro`.

8.2 Storage e schema

- Persistito in `preventivi_documenti` (kv_store JSONB).
- Per ogni documento: `id`, `sedeId`, `commessaId`, `nome`, `tipo`, `mimeType`, `size`, `storageKey?`, `checksum?`, `dataBase64?`, `note`, `statoAtUpload`, `createdBy`, `createdAt`.
- `storageKey` è la fonte canonica dopo la migrazione; `dataBase64` resta per record legacy/fallback. La lista `byCommessa` non restituisce i byte e usa `hasData`; `byId` rilegge dallo storage e ricostruisce il payload atteso dal client.

8.3 Upload — controlli

- **MimeType allowlist** (stored XSS hardening):
 - `application/pdf`
 - `image/jpeg`, `image/png`, `image/gif`, `image/webp`, `image/heic`, `image/heif`
 - `application/msword`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document`
 - `application/vnd.ms-excel`, `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`
 - **Esclusi esplicitamente** `text/html` e `image/svg+xml`.
- **Dimensione**: validata sul payload reale (lunghezza base64 decodificata, NON il campo `size` lato client). Cap 10 MB.
- Il `size` archiviato è quello calcolato dal server.

8.4 Auto-rename

- Se `keepNome === false`, il file in upload viene rinominato secondo `renameForStato({stato, cliente, originalName})` (es. "Misure esecutive Mario Rossi.pdf").
- Se `keepNome === true` (usato dai preventivatori), il nome viene preservato e solo deduplicato.
- Disambiguazione automatica: se il nome esiste già per la stessa commessa, viene appeso (2), (3), ecc.

8.5 Anteprima e download

- Anteprima inline in `<iframe>` per PDF (URL `blob:` derivato dal base64) o in `<img>` con zoom/rotate per immagini.
- Download via `<a download>`.
- Invio email via `mailto:` con corpo precompilato (no upload server-side dell'allegato).

8.6 Eliminazione

- Soft delete NON previsto. La cancellazione è definitiva.

8.7 Allegati ticket

- Pattern identico (router `ticketAllegati`), con `storageKey` /checksum e fallback base64, stessa allowlist mime, stesso size check, cap 10 MB.
 - Cancellando un ticket si cancellano in cascata i suoi allegati (`deleteAllegatiByTicket`).
-

9. Doc gate (gate documentale)

9.1 Regola

- Una transizione **in avanti** verifica che esista almeno un documento con uno dei tipi richiesti dallo stato CORRENTE (`REQUIRED_DOC_TIPI_PER_STATO`).
- Conta solo se il documento è stato caricato **mentre la commessa era in quello stato** (campo `statoAtUpload`), così un preventivo non può soddisfare un gate diverso.
- Per i documenti legacy senza `statoAtUpload`, fallback permissivo: il tipo è sufficiente.

9.2 Stati daily reminder

Per gli stati `aggiornamento_contratto`, `fatture_pagamento`, `da_ordinare` viene generata anche una notifica giornaliera (vedi §27.3) anche oltre la soglia di priorità.

9.3 Bypass "Procedi comunque"

- **Server.** `commesse.update` accetta un flag `force: boolean`. Se assente, il gate è bloccante; in caso di mancanza documenti il server lancia un errore con prefisso `DOC_GATE_BLOCKED:` e messaggio human-readable ("Non è stato caricato il file Procedere comunque?").
- **Client (scheda commessa).** Il pulsante "Avanza" non è più disabilitato dal gate. Al click, se il gate è violato, mostra un `ConfirmDialog` non-destructive con label "Procedi comunque". In conferma chiama `update({ stato, force: true })`.
- **Client (Kanban).** Onerror della mutation: se `err.message` inizia per `DOC_GATE_BLOCKED:`, lo stesso `ConfirmDialog` viene aperto e su conferma rilancia con `force: true`. Errori non-gate restano nel banner inline.
- **La state machine resta invariata.** `force` bypassa SOLO il gate documentale, non la shape delle transizioni.

9.4 Indicatore UI

Sotto l'header della commessa è presente una card con stato del gate:

- Verde se tutti i documenti richiesti sono presenti.
 - Ambra se mancano: lista di tipi richiesti con badge "Caricato"/"Mancante" + bottone "Carica file" che preimposta il tipo nel dialog upload.
-

10. Aperture (Rilievo)

- Una **apertura** è un singolo serramento all'interno di una commessa.
- Campi: `commessaId`, `codice`, `descrizione`, `piano`, `locale`, `tipologia` (`finestra` | `portafinestra` | `porta` | `scorrevole` | `fisso` | `altro`), `larghezza`, `altezza`, `profondita`, `materiale`, `colore`, `vetro`, `noteRilievo`, `criticitaAccesso`, `stato`.
- Stati apertura: `da_rilevare` → `rilevata` → `ordinata` → `consegnata` → `in_posa` → `posata` → `verificata`.
- Le aperture sono visibili nella scheda commessa e nella vista **RilievoDetail** dedicata.
- Una commessa **può** avere zero aperture (es. preventivo iniziale generico).

11. Board Kanban (/kanban)

11.1 Layout

Quattro **fasi**, ognuna con N colonne:

Fase	Colonne
Vendita	Preventivo, Misure Esecutive, Aggiornamento Contratto
Ordine & Produzione	Fatture / Pagamento, Da Ordinare, Produzione
Consegna & Posa	Richiesta Secondo Acconto, Attesa Posa
Chiusura	Finiture / Saldo, Interventi / Regolaz.

Lo stato `archiviata` non compare.

11.2 Card commessa

Mostra: codice, badge priorità, cliente, città, indicatore di consegna (data confermata o indicativa), pulsanti **Indietro** / **Avanza** con etichetta dello stato target. In più (v4):

- **Bordo sinistro 3 px** colorato per priorità (urgente rosso, alta ambra, media blu, bassa grigio).
- **Chip "fermo N gg"** quando la commessa non riceve update da ≥ 5 giorni (ambra) o ≥ 10 (rossa).
- **Blocco prodotti magazzino**: primi 2 prodotti con data consegna corta (✓ verde arrivato, rosso in ritardo) + "+N altri prodotti" (vedi §36).
- **Chip "Da saldare € N"** (rossa) nelle fasi `attesa_posa` / `finiture_saldo` / `interventi_regolazioni` quando il residuo pagamenti è > 0 (vedi §37).

11.2-bis Colonne con overflow

Ogni colonna mostra al massimo **5 card**; oltre compare il toggle tratteggiato "Mostra altre N" / "Mostra meno" per non forzare scroll infiniti.

11.3 Filtri

- Search per codice/cliente/città.
- Filtro priorità (Tutte/Urgente/Alta/Media/Bassa).

- Toggle "Nascondi vuote" / "Mostra vuote".
- Chip per fase (Tutte le fasi / per singola fase).
- Tutte le fasi possono essere collassate.

11.4 KPI bar

Quattro cards: **Attive**, **Urgenti**, **Alte**, **Consegne da confermare** (commesse in `produzione` senza `data-ConsegnaConfermata`).

11.5 Doc gate sul Kanban

Le frecce **Avanza** seguono lo stesso doc gate. Errore `DOC_GATE_BLOCKED:` → `ConfirmDialog` "Procedi comunque". Errori non-gate → banner inline rosso.

11.6 Esclusioni

- Le commesse soft-archivate non appaiono.

12. Calendario / Planning (/planning)

12.1 Viste

- **Mese** (*default*) — griglia 6×7 (la sesta settimana è renderizzata solo se il mese vi sconfina); oggi evidenziato con pill primary; giorni fuori mese e weekend attenuati; chip evento pieni (colore per tipo, testo bianco) con ora + nome cliente; overflow "+N altri" apre la vista giorno.
- **Settimana** — 7 colonne lun-dom; weekend leggermente attenuati.
- **Giorno** — singola colonna. Ordine switcher: Mese · Settimana · Giorno.

12.2 Tipi di intervento

`rilievo`, `posa`, `assistenza`, `altro` . Colori dedicati per tipo.

12.3 Card intervento (joined info)

Ogni card mostra:

- Ora inizio – ora fine.
- Tipo intervento.
- **Nome + cognome** del cliente (lookup commessa → cliente).
- **Indirizzo** (fallback `intervento.indirizzo` → `commessa.indirizzo` → `cliente.indirizzoLavoro` → `cliente.indirizzo`).
- Note brevi.
- Stato (`pianificato` di default).

12.4 Dialog dettagli appuntamento

In modalità modifica, sopra al form, viene mostrato un blocco di sintesi joined:

- Codice commessa, stato, badge priorità.

- Nome cliente.
- Indirizzo cantiere.
- Telefono (link `tel:`).
- Email (link `mailto:`).
- Squadra assegnata.
- Pulsante "Apri commessa" → naviga alla scheda commessa.

12.5 Auto-fill

Quando si seleziona una commessa in dialog di creazione, l'indirizzo viene auto-popolato dalla commessa (solo se il campo è vuoto, per non sovrascrivere override manuali).

12.6 Drag & drop

- Un intervento può essere trascinato fra giorni nelle viste settimana e mese.
- L'update della data avviene via `interventi.update({ dataPianificata })`.

12.7 Stati intervento

`pianificato`, `in_corso`, `completato`, `sospeso`, `annullato`. L'avvio imposta `dataInizio`; la chiusura imposta `dataFine`. Le entry `annullato` legacy sono **purgate** automaticamente al boot (cleanup nel `persistedStore.onLoad`).

12.8 Link entità

Un intervento può essere collegato a una delle entità: `commessa`, `ticket`, `reclamo`, `rifacimento`. Solo `commessa` è obbligatoria quando il `linkKind === "commessa"`.

12.9 Eventi Google (overlay read-only)

Gli eventi importati dai calendari Google (vedi §38.2) compaiono in tutte e tre le viste, ordinati per ora insieme agli appuntamenti CRM ma **in sola lettura**: stile distinto (badge GOOGLE + lucchetto, bordo sinistro nel colore della sorgente), nessun drag/edit/delete. Il click apre un dialog dettagli (data, orario/tutto il giorno, luogo, calendario di origine). Una legenda sopra la griglia elenca le sorgenti attive.

12.10 Card intervento (restyle v4)

Chip tipo pieno (POSA/RILIEVO/ASSISTENZA/ALTRO) su sfondo tinta + bordo sinistro 4 px nel colore del tipo; ora in mono grassetto. Nel dialog di modifica, accanto al telefono, è presente il bottone **WhatsApp** con messaggio di conferma appuntamento precompilato (vedi §41).

13. Ticket post-vendita (/ticket e contenuti di /reclami)

13.1 Modello

- Campi: `commessaId?`, `clienteId?`, `contatto?`, `aperturaId?`, `oggetto`, `descrizione`, `categoria`, `priorita`, `stato`, `solleciti[]`, `assegnatoA?`, `dataRisoluzione?`, `esitoIntervento?`, `apertoBy?`.

- Categorie: `difetto_prodotto`, `difetto_posa`, `regolazione`, `sostituzione`, `garanzia`, `altro`.
- Priorità: come per le commesse.
- `apertoBy` registra l'utente che apre il ticket (usato dai permessi di eliminazione, §13.6).

13.2 Ticket senza commessa

Una chiamata di assistenza arriva spesso **prima** che esista una commessa, e a volte prima ancora che il cliente sia a sistema. Perciò **solo l'oggetto è obbligatorio**; l'intestazione del ticket può essere data, in ordine di precisione, da:

1. **commessa** collegata (`commessaId`) — caso classico;
2. **cliente** già in anagrafica (`clienteId`), senza commessa;
3. **contatto** in testo libero (`contatto`), es. "Sig. Verdi 3401234567".

La UI mostra il primo disponibile, con badge "Senza commessa" quando manca, e un grigio "Senza cliente" se non c'è nulla. Il collegamento **può essere fatto in seguito**: il dialog di modifica espone commessa/cliente/contatto, e agganciando una commessa i campi più deboli vengono azzerati per non lasciare tre risposte in conflitto.

13.3 State machine ticket

`aperto` → `assegnato` → `in_lavorazione` → `chiuso`. Disponibile **rollback** di una posizione (clear `data-Risoluzione` se si esce da `chiuso`). Da `aperto` il rollback è rifiutato.

Lo stato **risolto è stato ritirato** (§13.7): fra risolto e chiuso non cambiava nulla nella pratica. Il backfill al boot converte i record esistenti; `updateStato` accetta ancora il valore legacy dai client vecchi e lo piega su `chiuso`. Stesso collasso applicato ai **reclami** (§14.1).

13.4 Solleciti

Registro `solleciti[]` sul ticket: { `data`, `nota?`, `utenteId` }. La procedure `ticket.sollecita` aggiunge una voce; è rifiutata sui ticket chiusi ("riaprilo prima di sollecitare"). In lista compare un badge ambra "**N solleciti · ultimo gg/mm**", così si vede a colpo d'occhio da quanto un ticket è fermo nonostante i solleciti. Il dialog mostra lo storico completo.

13.5 Interventi pianificati dal ticket

Bottone **Pianifica** sul ticket: data, ora inizio/fine, squadra, note. Crea un intervento di tipo `assistenza` già collegato (`ticketId`, `commessaId`, indirizzo ereditato dalla commessa) che appare nel Calendario e sotto il ticket con data, ora, squadra e stato — con "**Senza squadra**" evidenziato in colore warning quando manca.

13.6 Eliminazione

Possono eliminare: la **direzione, chi ha aperto** il ticket (`apertoBy`), o il **proprietario della commessa** collegata. Se la commessa è stata cancellata si ricade sul solo controllo direzione — diversamente il ticket sarebbe indistruttibile. Cancellando un ticket si cancellano in cascata i suoi allegati.

13.7 Ricerca e presentazione

- **Ricerca** su: nome cliente (da commessa o da `clienteId`), codice e città della commessa, oggetto, descrizione, id `TK-000N`, categoria, esito intervento e **note dei solleciti**.
- Il filtro per stato è **lato client** su tutta la lista di sede: i chip riportano i conteggi reali e la ricerca attraverso anche gli stati non selezionati (cercare un cliente non deve fallire perché il suo ticket è chiuso).
- **Card**: nome cliente in grassetto in testa, poi codice commessa (cliccabile) e `TK-000N`, quindi l'oggetto, le etichette, la descrizione in blocco espandibile (spesso è la nota importata da To Do), gli interventi collegati e infine il footer con meta e azioni. Barra colorata a sinistra: rossa se aperto ad alta priorità, verde se chiuso, neutra altrimenti.
- Empty state distinti fra "nessun ticket" e "nessun risultato per «...»", il secondo con azione **Azzera i filtri**.

13.8 Allegati ticket

Vedi §8.7.

14. Reclami e Rifacimenti (/reclami)

Pagina unificata che gestisce due entità correlate ma distinte.

14.1 Reclamo

- Campi: `commessaId`, `clienteNome`, `descrizione`, `responsabile?`, `stato`, `dataApertura`, `dataRisoluzione?`, `soluzione?`.
- Stati: `aperto`, `in_gestione`, `chiuso` (lo stato `risolto` è ritirato come per i ticket, §13.3; i record esistenti sono convertiti al boot).

14.2 Rifacimento

- Campi: `commessaId`, `clienteNome`, `descrizione`, `elemento`, `fornitoreCoinvolto?`, `ordineRifacimentoId?`, `costoStimato?`, `responsabilita (interna|esterna)`, `responsabile?`, `stato`, `dataApertura`, `dataChiusura?`.
- Stati: `aperto`, `in_gestione`, `in_produzione`, `completato`, `chiuso`.

15. Verbali (/verbale/:interventoId + VerbaleChiusura)

- Tipo: `chiusura_lavori` | `sopralluogo` | `consegna` (default `chiusura_lavori`).
 - Campi: `interventoId`, `commessaId`, `tipo`, `data`, `noteCliente?`, `noteTecnico?`, `firmaClienteData?`, `firmaTecnicoData?`, `firmaCliente`, `firmaTecnico`, `apertureCompletate`, `apertureTotali`, `anomalieRiscontrate`, `stato` (`bozza|firmato`).
 - Lo stato passa a `firmato` quando entrambe le firme sono presenti.
-

16. Anomalie

- Campi: `commessaId`, `aperturaId?`, `interventoId?`, `categoria`, `priorita`, `descrizione`, `risoluzione?`, `stato`, `segnalataBy?`, `risoltaBy?`, `risoltaAt?`.
 - Categorie: `materiale_difettoso`, `misura_errata`, `danno_trasporto`, `difetto_posa`, `problema_accessorio`, `non_conformita`, `altro`.
 - Priorità: `bassa`, `media`, `alta`, `critica`.
 - Stati: `aperta`, `in_gestione`, `risolta`, `chiusa`. Endpoint dedicato `resolve` imposta `stato=risolta`, `risoluzione`, `risoltaAt`.
-

17. Garanzie (`/garanzie` , direzione-only)

- Tipi: `prodotto`, `posa`, `accessorio`, `vetro`, `altro`.
 - Campi: `commessaId`, `aperturaId?`, `tipo`, `descrizione`, `fornitore?`, `dataInizio`, `durataMesi`, `dataScadenza`, `stato`, `documentoRif?`, `note?`.
 - `dataScadenza` calcolata server-side da `dataInizio` + `durataMesi`.
 - Stati: `attiva`, `scaduta`, `sospesa`, `revocata`.
 - Statistiche: totale, attive, in scadenza (entro 90 giorni), scadute.
-

18. Squadre (`/squadre` , direzione-only)

- Campi: `nome`, `caposquadra?`, `telefono?`, `note?`, `attiva`.
 - Le squadre attive appaiono nei dropdown della scheda commessa, dialog intervento, calendario.
 - L'inattivazione (toggle `attiva`) preserva storico ma rimuove la squadra dai picker.
-

19. Fornitori (`/fornitori` , direzione-only)

19.1 Anagrafica fornitore

- Campi: `ragioneSociale`, `partitaIva`, `indirizzo?`, `citta?`, `telefono?`, `email?`, `categoria`, `referenteCommerciale?`, `scontistica?`, `note?`, `attivo`.
- Categorie: `pvc`, `alluminio`, `vetro`, `ferramenta`, `persiane`, `blindati`, `accessori`, `guarnizioni`, `altro`.

19.2 Ordini fornitore

- Campi: `fornitoreId`, `commessaId`, `codiceOrdine`, `stato`, `dataOrdine`, `dataConsegnaPrevista?`, `dataConsegnaEffettiva?`, `righe[]`, `noteOrdine?`, `noteRicevimento?`, `importoTotale`.
- Stati: `bozza`, `inviato`, `confermato`, `in_transito`, `ricevuto_parziale`, `ricevuto`, `contestato`.
- Riga ordine: `descrizione`, `codiceArticolo?`, `quantita`, `quantitaRicevuta`, `unitaMisura`, `prezzoUnitario?`, `lotto?`, `conforme?`, `noteDifetto?`.
- L'update di stato accetta `righeAggornate` per registrare la ricezione parziale.

19.3 Listini fornitore

- Campi: `fornitoreId`, `nome`, `versione`, `dataValidita`, `nomeFile`, `tipo` (`pdf|excel|altro`), `note?`.
 - I listini sono solo metadati: il file effettivo viene gestito altrove (cartella condivisa o sezione documenti separata).
-

20. Produzione (/produzione , direzione-only)

Tre sotto-router: `bom` (distinte base), `fasi`, `nc` (non conformità).

20.1 Distinte base (BOM)

- Campi: `commessaId`, `aperturaId`, `stato`, `componenti[]`, `noteValidazione?`, `validataDa?`, `dataValidazione?`.
- Componenti: `tipo` (`profilo|vetro|ferramenta|guarnizione|accessorio`), `descrizione`, `codiceArticolo?`, `fornitoreId?`, `quantita`, `unitaMisura`, `lotto?`, `note?`.
- Stati BOM: `bozza`, `validata`, `in_produzione`, `completata`.
- Validazione consentita solo da `bozza` → `validata`.

20.2 Fasi produzione

- Campi: `commessaId`, `aperturaId?`, `distintaBaseId`, `fase`, `ordine`, `stato`, `operatore?`, `dataInizio?`, `dataFine?`, `checklistItems[]`, `note?`.
- Stati fase: `da_fare`, `in_corso`, `completata`, `non_conforme`.
- Checklist item: `descrizione`, `obbligatorio`, `completato`, `esito?` (`ok|non_conforme`), `note?`. Toggle dedicato `toggleChecklist`.
- `dataInizio` impostata su `in_corso`; `dataFine` su `completata`.

20.3 Non conformità (NC)

- Campi: `commessaId`, `aperturaId?`, `faseProduzioneId?`, `tipo`, `gravita`, `descrizione`, `azioneCorrettiva?`, `stato`, `segnalataDa`, `dataApertura`, `dataChiusura?`.
 - Tipi: `materiale_difettoso`, `errore_taglio`, `errore_assemblaggio`, `vetro_rotto`, `ferramenta_errata`, `altro`.
 - Gravità: `lieve`, `media`, `grave`, `bloccante`.
 - Stati: `aperta`, `in_gestione`, `risolta`, `chiusa`. La chiusura imposta `dataChiusura`.
-

21. Preventivatori (/preventivatori)

Pagina hub con cards per azienda. L'accesso è aperto a tutti gli utenti autenticati.

21.1 Fivizzanese — Persiane (/preventivatori/fivizzanese/persiane)

Calcolo step-by-step:

1. **Selezione modello** (catalogo `shared/listini/fivizzanese.ts`).
2. **Quantità persiane standard** (con prezzo base modello).
3. **Quantità persiane con centinatura** (prezzo extra).
4. **Posa** (preset).
5. **Smontaggio vecchie** (opzionale, prezzo per unità).
6. **Promo** quando applicabile:
 - **Promo "RAL 6005 Opaco"**: prezzo aggiornato a **210 €** (precedentemente 200 €).
 - Quando la promo è attiva, viene aggiunto un **ricarico del 50%** sul totale persiane.
7. **IVA** (10% o 22%) — selettore.
8. Output:
 - Riepilogo a video con righe: totale persiane, supplementi, centinatura, ricarico promo, smontaggio, imponibile, IVA, totale lordo.
 - Esportazione **PDF** con `jspdf-autotable`; salvataggio automatico come documento `preventivo` sulla commessa (`keepNome=true`; nome file `"Preventivo {cliente} - Fivizzanese.pdf"`).

21.2 Punto del Serramento — Persiane (`/preventivatori/punto-del-serramento/persiane`)

Stesso schema più:

- **Sconto** percentuale modificabile, **max 30%** (clamp server-side e client-side).
- **IVA** (10% o 22%) — selettore.
- Output PDF analogo con righe `lordo, sconto, imponibile, IVA, totale`.

22. Archivio (`/archivio`)

22.1 Soft-archive

- Endpoint `commesse.archive(id)`: imposta `archivedAt = ISO now`.
- Endpoint `commesse.restore(id)`: imposta `archivedAt = null`.
- Lo stato (`stato`) e i collegamenti (file, aperture, interventi, prodotti) **non vengono toccati**. Restore = la commessa torna esattamente com'era.

22.2 Visibilità

- Le commesse soft-archivate sono escluse da:
 - `commesse.list` (scope default `exclude`).
 - `commesse.stats` e `commesse.byPriorita`.
 - Board Kanban.
 - Dashboard.
 - Notifiche proattive (vedi §27).
- Restano accessibili a `/archivio` (scope `only`) e tramite link diretto `/commesse/:id`.

22.3 Pagina Archivio

- Lista con search per codice/cliente/città/indirizzo.
- Per ogni riga: codice, stato originale (badge), priorità, cliente, indirizzo, data apertura, data archiviazione, pulsanti **Ripristina** e **Apri scheda**.

- Conferma esplicita prima del ripristino.

22.4 Pulsanti nella scheda commessa

- **Archivia** (se non archiviata) — conferma non-destructive.
 - **Ripristina** (se archiviata).
 - Banner di stato "Commessa archiviata" in alto quando applicabile.
-

23. Classifica venditori — RIMOSSA (16/07/2026)

La pagina `/classifica`, l'endpoint `commesse.classificaVenditori` e la voce di sidebar sono stati rimossi su richiesta della direzione. La sezione resta come riferimento storico (v3); il ranking commerciali non fa più parte del prodotto.

24. Utenti (`/utenti` , direzione-only)

24.1 Funzionalità

- Lista utenti con filtro per ruolo + search testuale.
- Creazione, modifica, eliminazione (tutte `adminProcedure`).
- Multi-ruolo (1–3 ruoli per utente).
- Toggle `attivo/inattivo` .
- Password gestita solo lato server (hashing scrypt, vedi §3.3). Il campo `hasPassword` (bool) è restituito al client al posto del contenuto.

24.2 Gating

- Rotta `/utenti` protetta da `<RequireDirezione>` .
- Sidebar mostra la voce **Utenti** solo a utenti `direzione` .

24.3 Email

- Univocità email enforced lato server (`utenti.create` rifiuta duplicati case-insensitive).
-

25. Notifiche proattive (dropdown header) — v2 personalizzate

25.1 Personalizzazione

Le notifiche sono calcolate on-demand **per l'utente corrente**: "owner" = `commessa.assegnatoA === userId` (fallback legacy `createdBy` quando `assegnatoA` è nullo). La `direzione` vede anche le notifiche di scope sede. Tutto è filtrato sulla sede attiva.

25.2 Fonti

#	Fonte	Destinatari	Severità	Note sull'id
1	Priority aging — commessa ferma oltre soglia (bassa 7 gg, media 5, alta 3, urgente 1)	owner	per priorità	embedde <code>updatedAt</code> : "segna letta" tace finché la commessa non si muove di nuovo
2	Daily reminder su stati bottleneck (<code>aggiornamento_contratto</code> , <code>fatture_pagamento</code> , <code>da_ordinare</code>)	owner	escalation per età (≥5 urgent, ≥3 warning)	embedde la data: letta oggi, rispunta domani
3	Stato → ruolo (<code>da_ordinare</code> →ordini, <code>misure_esecutive</code> →tecnico_rilievi, <code>fatture_pagamento</code> / <code>finiture_saldo</code> →amministrazione)	utenti col ruolo	info	id = (commessa, stato): letta finché lo stato non cambia
4	Consegna da confermare — <code>produzione</code> senza <code>dataConsegnaConfermata</code>	owner + direzione	warning	id stabile per commessa
4b	Saldo residuo nelle fasi finali (<code>attesa_posa</code> , <code>finiture_saldo</code> , <code>interventi_regolazioni</code>) con <code>importoTotale</code> impostato e residuo > 0	owner + direzione + amministrazione	warning	id embedde il residuo: un incasso parziale ri-notifica
5	Garanzie scadute (urgent) o in scadenza ≤30 gg (warning)	amministrazione + direzione	urgent/warning	id per garanzia
6	Ticket aperti/assegnati su commesse possedute	owner + direzione	da priorità ticket	id ruota con lo stato: riapertura ri-notifica
7	Interventi di oggi/domani senza squadra	owner (direzione per i non collegati)	warning oggi / info domani	id per (intervento, data)

25.3 Stato lettura persistito

- Store `notifiche_read` : una riga per utente { `userId` , `readIds[]` } (cap 800 id, FIFO).
- `notifiche.markRead({ids})` e `notifiche.markAllRead()` .
- `notifiche.list` ritorna `read: boolean` ; ordinamento: non lette prima, poi severità (urgent→warning→info), poi recency. Cap 100.
- `notifiche.count` = **solo non lette** (badge campanella: rosso se contiene urgenti, ambra altrimenti).

- Ogni notifica porta un campo `link` (commessa, `/ticket`, `/garanzie`, `/planning`): il click nel dropdown segna letta e naviga.

25.4 Dropdown

Header con conteggio "N da leggere — personalizzate per te" + bottone "Segna tutte lette". Icone per tipo (camion=consegna, ticket, scudo=garanzia, calendario=intervento). Le lette sono attenuate in fondo; pallino primary sulle non lette.

25.5 Esclusioni

Come v3: niente notifiche per commesse `archiviata` o soft-archivate.

26. Dashboard (/)

26.1 Header personale

"Ciao {nome} — ecco la tua giornata" + data lunga it-IT.

26.2 "Da fare oggi" — feed azioni personalizzato

- **Scope:** direzione vede tutta la sede (etichetta "Tutta la sede"); gli altri solo le proprie commesse (`assegnatoA`, fallback `createdBy`) — etichetta "Le tue attività".
- Fonti, ordinate per urgenza e cap a 8 voci:
 1. Interventi di **oggi senza squadra** (CTA "Apri calendario").
 2. Commesse **urgenti** / ticket urgenti / garanzie scadute.
 3. **Da incassare € N** — residuo pagamenti nelle fasi finali.
 4. **Consegne da confermare** (CTA "Conferma consegna").
 5. Ticket aperti sulle proprie commesse.
 6. Garanzie in scadenza 30 gg (direzione/amministrazione).
- Stato vuoto esplicito: "Niente da fare per ora ..." con icona verde (la card non sparisce).

26.3 KPI principali

Cards Commesse attive, Urgenze, Consegne da confermare, Ticket aperti (+ Interventi settimana). Zero = card "spenta" non cliccabile; >0 = accent bar + navigazione alla lista filtrata. Polling live.

26.4 Calendario settimanale

Slot 7 giorni con eventi per tipo (filtri per calendario) + navigazione settimana.

27. Integrazioni esterne (/integrazioni = Impostazioni)

La pagina Impostazioni ospita l'hub **Gestione** (direzione-only: Fornitori, Produzione, Squadre, Garanzie, Preventivatori) e le card integrazioni:

27.1 Backup notturno su Google Drive

Vedi §39. Card con stato collegamento, ultimo backup, prossima esecuzione, "Esegui ora", collega/scollega account.

27.2 Fatture in Cloud → Clienti

Vedi §40. Card OAuth con stato collegamento, scadenza credenziali, selettore azienda, switch abilitazione, "Sincronizza ora" e disconnessione. Il token manuale mascherato resta dentro un pannello secondario come fallback di emergenza.

27.3 Mostra Google nel calendario CRM (import)

Vedi §38.2. Gestione sorgenti: nome + indirizzo iCal segreto (mascherato in lista), colore auto-assegnato, toggle mostra/nascondi, stato sync, "Aggiorna", istruzioni passo-passo.

27.4 Pubblica il calendario CRM su Google (export)

Vedi §38.1. Elenco feed copiabili (Tutti/Rilievi/Pose/Assistenza/Altro) + "Rigenera token" con avviso di revoca.

27.5 Microsoft To Do

Card presente ma **mock** (non ancora collegata a Graph API). Storicamente il flusso operativo viveva su liste To Do; la migrazione del 15/07/2026 (vedi §43) le ha importate nel CRM.

27.6 Notifiche owner / Google Maps

Invariati da v3: `system.notifyOwner` (Manus Notification Service, admin-only) e componente `<Map/>` via proxy interno.

27.7 Roadmap: Antenore (Wnd/Oknoplast)

Integrazione richiesta al fornitore (import automatico clienti/preventivi/commesse/ordini creati sul loro CRM). In attesa di risposta tecnica sugli endpoint disponibili; prevista sync unidirezionale Antenore → Ruffino Ops con dedup su id.

28. Persistenza

28.1 KV store

- Tabella `kv_store`(key text primary key, data jsonb, updated_at timestampz).
- Ogni router business possiede una o più raccolte persistite, tra cui `clienti`, `commesse`, `tickets`, `ticket_allegati`, `preventivi_documenti`, `utenti`, `sedi`, `backup_*`, `fic_config`, `fic_fatture`, `caselle_email`, `whatsapp_*`, `azioni_suggerite`, `conoscenza_aziendale`, `agente_esecuzioni`, `agente_config` e `tars_chat`.

La tabella `comunicazioni` è separata dal KV store: insert idempotente per `(casella_id, canale, message_id)`, indici per lista e tombstone per le eliminazioni dal CRM (§51).

Il refresh token Google del backup è inoltre **specchiato su file** (`data/backup-oauth.json`, mode 600, gignored) così i riavvii senza DATABASE_URL non scollegano Drive; la riga DB, quando presente, ha precedenza.

28.2 Lifecycle

- **Bootstrap.** All'avvio `bootstrapAll()` legge ogni raccolta. Tre stati possibili per la singola key:
 - `firstBoot = true`: nessuna row presente → callback `onLoad` può popolare seed.
 - `firstBoot = false, loaded = true`: row presente, dati ripristinati con `reviver Date`.
 - `firstBoot = false, loaded = false`: errore transiente; **i save sono bloccati** finché la background recovery non riesce a leggere lo stato dal DB.
- **Save.** Debounciato 200 ms; usa `sql.json()` per evitare doppia encoding. Retry exponential backoff su errori transienti (EAI_AGAIN, ENOTFOUND, ECONNREFUSED, ECONNRESET, ETIMEDOUT, ENETUNREACH, EHOSTUNREACH, EPIPE).
- **Shutdown.** Handler `SIGTERM / SIGINT` chiamano `flushAll()` per non perdere mutazioni in flight, poi chiudono la pool.

28.3 Recovery

- Se `ensureSchema` o `load` falliscono dopo i retry → `backgroundRecover` (max 30 tentativi a intervalli di 5 s) finché ogni raccolta non risulta `loaded = true`. Nessun seed viene applicato in caso di fallimento (per non sovrascrivere dati reali con un seed accidentale).

28.4 Legacy double-encoded payload

- Riconosciuto a load: se `data` è stringa JSON, viene parsata e schedulata una riscrittura corretta.

28.5 Vincoli su Postgres

- Connessione Postgres-js: max 5 connessioni, `idle_timeout=20s`, `connect_timeout=30s` (Railway internal DNS warmup).
- SSL automatico quando l'URL contiene `sslmode=require` o l'host Railway.

29. UI/UX e componenti

29.1 Sistema

- Tailwind + shadcn/ui (Button, Card, Badge, Dialog, AlertDialog, Avatar, Input, Label, Select, Switch, Tabs, Tooltip, DropdownMenu).
- Icone lucide-react.
- Toast: sonner.
- Font Plus Jakarta Sans; palette chiara calda con superfici bianche, inchiostro scuro e giallo come accento. I colori di stato usano token semantici (`success`, `warning`, `danger`, `info`), non hex locali.
- Raggi tra 8 e 14 px, bordi visibili e ombre contenute. Le superfici operative non devono apparire fredde, eccessivamente opache o spigolose.

- Componente `<SearchSelect>` riutilizzato per dropdown ricercabili (utenti, squadre, fornitori, ecc.).
- Componente `<ConfirmDialog>` riutilizzato per: cancellazioni, archiviazioni, "Procedi comunque" del doc gate, ripristino archivio.
- Componente `<FilePreviewDialog>` per preview PDF/immagini.

29.2 Sidebar

- Voci dinamiche; quelle con `direzioneOnly` filtrate per ruolo.
- Resizable (larghezza personalizzabile da utente).
- Avatar utente in basso con menù: nome, email, Esci.

29.3 Mobile

- Layout responsivo. Sidebar collassata in modalità mobile; nasconde scritte tenendo solo icone.
- **Header delle schede** (commessa, cliente): su viewport `< sm` il titolo e la riga di azioni si impilano (`flex-col` → `sm:flex-row`) e i bottoni vanno a capo (`flex-wrap`) invece di uscire dallo schermo.
- **Tabelle di lista** (Commesse, Clienti): le colonne secondarie sono nascoste progressivamente con `hidden {sm|md|lg|xl}:table-cell`, così su telefono restano solo le essenziali — Codice/Cliente/Stato per le commesse, Nome/Telefono per i clienti — con padding ridotto (`px-3`) sulle colonne mantenute. **Non** viene usato un wrapper `overflow-x-auto` : creerebbe un contenitore di scroll che romperebbe gli header sticky (regressione già occorsa e corretta in passato).
- Board, Calendario, Dashboard, Pagamenti e Magazzino riflowano nativamente (griglie responsive + `flex-wrap`); nessuno scroll orizzontale di pagina.
- Le tabelle delle sezioni direzione-only a bassa frequenza (Fornitori, Produzione) restano larghe: sono pensate per l'uso da desktop.

29.4 Empty states

- Tutte le pagine principali hanno empty state esplicito con istruzioni sul prossimo passo.

30. Errori e telemetria

30.1 Convenzioni errori

- Gli errori non-tRPC sono ritornati come `Error` con messaggio human-readable in italiano.
- Errori "marker" usano un **prefisso** come `DOC_GATE_BLOCKED:` che il client identifica per offrire UI dedicata.

30.2 Logging

- Tutto il logging della persistenza passa da `console.log/warn/error` con prefisso `[persistence]` .
 - I save e i load mostrano sempre il conteggio degli elementi per raccolta.
-

31. Roadmap aperta (lavori noti)

31.1 Sicurezza

- **Operatività:** ruotare le vecchie credenziali seed eventualmente ancora usate, rifare il login locale `gh` e revocare token GitHub non riconosciuti.
- **Decisione separata:** purge delle password seed dalla cronologia (`BFG/git filter-repo`). Richiede riscrittura SHA, force-push concordato e riallineamento di ogni clone.
- CSP (Content Security Policy) tarata su Vite, blob preview, Maps proxy.

31.2 Ottimizzazione

- **Attivazione object storage:** il layer per-file è completo; restano configurazione R2 su Railway, probe, backup Drive, dry-run sui dati reali e apply (§47). Il dry-run locale senza `DATABASE_URL` non è una verifica della produzione.
- Aggregato dashboard in un unico endpoint per ridurre il fan-out lato client.
- Monitoraggio del rapporto `cache_read_input_tokens` /input token e del costo per trigger Tars dopo il deploy (§50).

31.3 UX

- Drag&drop diretto sulle colonne del Kanban (oggi solo bottoni avanza/indietro).
- Confetti hardware-accelerati (opzionale).

31.4 Integrazioni

- **Fatture in Cloud OAuth:** codice completato. Restano configurazione delle variabili Railway, registrazione del redirect e collegamento di ogni sede (§40.3).
- **Antenore (Wnd/Oknoplast):** connettore import clienti/preventivi/ordini — in attesa di specifiche dal fornitore.
- Esportazione CSV/Excel commesse, clienti, anomalie.
- Push notifications mobile.
- UI di restore dal backup Drive.

32. Glossario

- **Commessa.** Progetto specifico di vendita+installazione per un cliente.
- **Apertura.** Singolo serramento (finestra, porta, ...) all'interno di una commessa.
- **Stato.** Posizione corrente della commessa nella state machine.
- **Soft-archive.** Stato secondario, ortogonale allo stato del workflow: la commessa è nascosta dalle viste operative ma tutto è preservato.
- **Doc gate.** Vincolo per cui un avanzamento di stato richiede l'upload di documenti previsti per lo stato corrente.
- **Bypass.** Conferma esplicita dell'operatore tramite `force: true` per superare il doc gate.
- **Tier (classifica).** Un gruppo di venditori con lo stesso conteggio commesse; condividono il rank.
- **Pari merito.** Più persone con lo stesso conteggio. Condividono il rank e il gradino del podio.
- **Indirizzo di residenza.** Indirizzo fiscale del cliente (uso amministrativo).

- **Indirizzo di lavoro.** Indirizzo del cantiere (uso operativo per commesse, calendario, mappe).
 - **Tipo detrazione.** Modalità fiscale richiesta dal cliente: `ecobonus` o `ristrutturazione`.
-

33. Cronologia significativa

- **v4.9 (19/08/2026)** - Tars migrato a OpenAI Responses API con `gpt-5.6-sol` per le richieste umane, `gpt-5.6-terra` per gli automatismi, prompt caching per profilo/modello e chiusura forzata dopo il budget strumenti (§50-51).
- **v4.8 (19/08/2026)** - Coda Comunicazioni Tars resa lossless: continuazione automatica oltre 10 mail, trigger concorrenti preservati, retry API non annullabile, recupero periodico e dopo bootstrap, riattivazione da config/budget e stato d'attesa spiegato nella UI (§50-51).
- **v4.7 (19/08/2026)** - Ogni nuova email passa dalla classificazione strutturata di Tars; il filtro locale fornisce solo segnali, le esclusioni richiedono confidenza alta, i dubbi restano visibili e le mail saltate vengono ritentate (§50-51).
- **v4.6 (18/08/2026)** — Richieste di preventivo e opportunità protette da header spam e regole mittente; newsletter inutili ricondotte allo spam; Tars chiede l'assegnatario prima di proporre cliente e commessa e conserva il contesto nel seguito (§50-51).
- **v4.5 (18/08/2026)** — Comunicazioni con triage multilivello, filtro anti-spam/offerte prima dell'AI, regole mittente, azioni multiple e gestione Tars della singola mail con creazione lead approvata (§50-51).
- **v4.4 (18/08/2026)** — Tars con quadro aziendale, lettura controllata di organizzazione/produzione/qualità/documenti, audit periodico dei processi, proposta di miglioramenti misurabili, deduplica per chiave d'azione e Inbox operativa (§50).
- **v4.3 (14/08/2026)** — Inbox Comunicazioni email/WhatsApp (§51); Tars con fascicolo commessa, profili tool, prompt caching e cache per run (§50); OAuth FiC Authorization Code con refresh (§40.3); backup Drive compatibile con `storageKey`; probe e runbook R2 (§47); bootstrap utenti senza password fisse; sistema visuale caldo e code splitting (§52).
- **v4.2 (06/08/2026)** — Storage documenti su object storage con migrazione verificata (§47); marginalità e registro costi fornitore dentro la commessa (§45); post-vendita v2: solleciti, interventi pianificabili dal ticket, ticket senza commessa, ricerca, stato `risolto` ritirato (§13); squadre di posa visibili e assegnabili alla commessa (§46); prodotti dichiarati in creazione e modificabili dopo (§44); data di apertura in scheda e in lista (§9); formato e parsing unici degli importi (§48); correzioni notevoli (§49).
- **v4.1 (23/07/2026)** — Pagina Pagamenti (§37.4) con registrazione rapida degli acconti; acconti modificabili in place (§37.1-37.2); date programmabili sugli step della timeline, pensate per l'Appuntamento Posa (§35.2-bis); Magazzino a 2 tile per riga con 4 prodotti visibili, badge fornitore e filtro fornitore a tendina (§36.3); form cliente con Ragione sociale e Sede legale per i non privati (§5.2); responsive mobile su header schede e tabelle di lista (§29.3).
- **v4.0 (16/07/2026)** — Rimossa la Classifica venditori (§23). Multi-sede con isolamento completo; redesign UI (board v2, calendario mese-default con chip pieni, timeline note post-it, dashboard personalizzata); Magazzino (tile+popup, ordini, lead time, dropdown fornitori); registro acconti; notifiche v2 con stato lettura; sincronizzazione Google Calendar export+import; backup notturno Google Drive (OAuth drive.file); Fatture in Cloud sync clienti; WhatsApp deep link; scheda cliente PDF; hardening (scrypt versionato, CSRF, SSRF guard, trust proxy, mascheramento segreti); migrazione dati 2026 (§43).
- **v3.0** — Riallineamento completo del PRD al codice corrente: dual address, tipoDetrazione, dataConsegnaIndicativa, soft-archive, Archivio, Classifica venditori, doc-gate con bypass, hardening sicurezza (scrypt, JWT fail-hard, mimeType allowlist, rate-limit login, security headers, session eviction, logout server-side), assegnazione utente modificabile, preventivatori Fivizzanese e Punto del Serramento, Planning con joined info.

- v2.x — Versione precedente (PDF allegato in repo come riferimento storico).
 - v1 — Documento originale.
-

Parte II — Moduli introdotti dopo la v3

34. Multi-sede

34.1 Modello

- Store `sedi`: { `id`, `nome`, `indirizzo?`, `citta?`, `attiva` } (seed prima sede "La Spezia").
- Ogni entità business porta `sedeId` (backfill = 1 per i record pre-esistenti).
- Gli utenti hanno `sediIds`: `number[]` (accesso multi-sede).

34.2 Risoluzione della sede attiva

- Cookie/claim `active_sede` → `ctx.sedeId` su ogni richiesta tRPC.
- **SedeSwitcher** nella sidebar (in alto): cambia la sede attiva; tutte le query si rifanno sul nuovo scope.

34.3 Isolamento

- Ogni `list` filtra per `sedeId`; ogni mutation su entità esistente passa da `assertSedeScope` (404 anche in caso di id valido di altra sede — nessun information leak).
- Entità figlie (documenti, allegati, timeline, magazzino) ereditano lo scope dalla commessa/ticket padre.
- Pagina `/sedi` (direzione-only) per creare/gestire sedi; assegnazione `sediIds` dalla pagina Utenti.

35. Timeline ordine (scheda commessa)

35.1 Struttura

18 step fissi per commessa (store `timeline_steps`, creati lazy alla prima lettura), raggruppati nelle 4 fasi del board: Rilievo Misure, Firma Contratto, Fatturazione, Invio Fattura, 1° Acconto, Ordine Merce, Conferma Ordine, Acconto Fornitore, Data Spedizione Prevista, Pagamento Merce Pronta, 2° Acconto, Data Consegna Merce, Appuntamento Posa, Lista Merce Posata, DDT Posa, Finiture, Saldo, Recensione.

35.2 Interazione

- Barra avanzamento ($N/18 + \%$). Fasi collassabili; **la fase con lo step corrente e quelle contenenti note si aprono da sole.**
- Step corrente evidenziato (sfondo primary tenue) con bottone **Completa** one-click. Dialog di modifica per data, utente esecutore (SearchSelect) e note; step completato riapribile.
- Campi step: `stato` (`da_fare`|`in_corso`|`completato`), `dataCompletamento`, `utente`, `note`, `allegato?`.

35.2-bis Date programmate (appuntamenti futuri)

Il dialog dello step espone un campo «**Data (appuntamento o completamento)**» e due azioni distinte:

- «**Salva data**» — memorizza data/utente/note **senza** completare lo step. Serve a programmare in anticipo, tipicamente l'**Appuntamento Posa**, ma vale per qualsiasi step futuro.
- «**Segna come completato**» — completa lo step usando la data scelta (non più forzata a oggi).

Uno step non completato con data valorizzata mostra in riga una scritta blu « gg/mm/aaaa · utente», visivamente distinta dai metadati grigi degli step già completati.

35.3 Note come cittadino di prima classe

- Le note renderizzano come **post-it ambra** (icona + 12 px, `whitespace-pre-line`: le note multiriga migrate dai To Do conservano a-capo e separatori "— — —").
- L'header di ogni fase mostra un badge ambra " N" con il conteggio note.
- Le date di completamento sono formattate it-IT.

36. Magazzino (/magazzino)

36.1 Scope

Prodotti fisici in arrivo/da magazzino **per commessa**. Eleggibili solo commesse **da produzione in poi** (incluso; escluse archiviate) — gate applicato sia lato server (create) sia nel filtro pagina.

36.2 Modello (magazzino_prodotti)

```
{ id, sedeId, commessaId, nome, quantita, fornitore?, numeroOrdine?, dataOrdine?, dataConsegna?, arrivato, note?, createdAt, updatedAt }.
```

- **Fornitore** da dropdown fisso aziendale: Wnd, Oknoplast, Alias, Pail, Primed, HenryGlass, Palmieri, Errecci, Fivizzanese, Oskura, Korus, Punto del Serramento, Kopern, Citea, Cerrato, Brianzatende, Seraplastic, St Scale, Sharknet (valori legacy liberi restano selezionabili).
- **Lead time** = giorni `dataOrdine` → `dataConsegna`, mostrato per prodotto ( N gg) e come **KPI medio** sugli arrivati.
- Cascade: eliminando una commessa si eliminano i suoi prodotti.

36.3 UI

- **KPI**: Prodotti, In arrivo, Arrivati, Lead time medio.
- **Strip "Prossime consegne"**: le 5 consegne pendenti più vicine cross-commessa (rosse se scadute); il click apre il popup della commessa.
- **Ricerca + filtri**: chip di stato (Tutte / In arrivo / In ritardo / Arrivati) affiancati da un **menu a tendina dei fornitori** («Tutti i fornitori» + i 19 dell'elenco) che filtra le commesse contenenti almeno un prodotto di quel fornitore. Il vecchio chip «Con prodotti» è stato sostituito da questo dropdown.
- **Griglia di tile (2 per riga su desktop, 1 su mobile)**: codice, StatoChip, cliente (17 px bold), città, **primi 4 prodotti** colorati per stato (✓ verde arrivato, rosso in ritardo con data corta, grigio in arrivo) ciascuno con **mini-badge fornitore** accanto alla data, "+N altri prodotti", badge "N/M arrivati". Bordo rosso (ritardi) / verde (tutto arrivato). Ordinamento per urgenza (ritardi → consegna più vicina).

- **Popup dettaglio** (click sul tile, `max-w-6xl`): header con codice/cliente/stato/città + "Apri commessa" + badge; righe prodotto **tutte editabili inline** (quantità, fornitore, n° ordine, date, switch arrivato, nota ghost click-to-edit con blur-save); form "Aggiungi prodotto" su due righe.

36.4 Board

Le card del Kanban mostrano il blocco prodotti (vedi §11.2).

37. Pagamenti — registro acconti

37.1 Modello

Su ogni commessa: `importoTotale` (pattuito) + `pagamenti[]` embedded: { `id`, `importo`, `data?`, `metodo?`, `note?`, `createdAt` } con `metodo` ∈ `bonifico` | `contanti` | `assegno` | `pos` | `finanziamento` | `altro`.

- `importoIncassato` è **sempre ricalcolato dal server** come somma del registro (`addPagamento`, `updatePagamento`, `removePagamento`) → board, dashboard e notifiche coerenti senza duplicare logica.
- Gli acconti registrati sono **modificabili**: `updatePagamento` accetta importo, data, metodo e nota di una singola riga.
- Backfill: record legacy con incassato secco → unico acconto "Importo importato".

37.2 Card "Pagamenti" (scheda commessa)

- Totale pattuito editabile inline (blur-save), barra "% incassato · € N", **Residuo** grande (ambra finché > 0, verde a saldo).
- Registro acconti ordinato per data: data, importo bold, badge metodo, nota, **matita** e cestino. La matita trasforma la riga in un form inline (data, importo, metodo, nota + Salva/Annulla); al salvataggio residuo, barra, chip del board, dashboard e notifiche si aggiornano da soli.
- **Chips rapide 50% / 40% / 10%** del totale (il piano di pagamento tipico), cappate al residuo: un click precompila l'importo nel form di registrazione (data default oggi).
- Accent warning sulla card finché c'è residuo.

37.3 Propagazione

- Board: chip "Da saldare € N" nelle fasi finali (§11.2).
- Dashboard: voce "Da incassare € N" nel feed personalizzato (§26.2).
- Notifiche: fonte 4b (§25.2) — l'id embedde il residuo, un incasso parziale ri-notifica il nuovo valore.

37.4 Pagina Pagamenti (/pagamenti)

Vista cassa di sede, in sidebar dopo Magazzino. Mostra solo commesse attive (no archiviate).

- **KPI**: Pattuito · Incassato · **Da incassare** · numero commesse senza importo pattuito.
- **Strip «Ultimi incassi»**: gli acconti più recenti della sede (endpoint `commesse.pagamentiRecenti`, che appiattisce i registri ordinandoli per data di registrazione e resta sede-scoped); il click apre la commessa.
- **Righe ordinate per residuo decrescente** (i soldi più grossi in cima): codice, cliente, StatoChip, pattuito, barra percentuale con incassato (nascosta sotto `md`), **Residuo** in ambra oppure «Saldata» in verde.

- **Bottone «Acconto»** su ogni riga con residuo: apre un dialog rapido con chips **50/40/10%** + **«Salda tutto»**, data (default oggi), metodo e nota — registra senza dover aprire la commessa (riusa `addPaging`).
- **Filtri:** Con residuo (*default*) / Saldate / Senza importo / Tutte, più ricerca per codice o cliente.

38. Sincronizzazione Google Calendar

38.1 Export (CRM → Google) — feed iCal per sede

- Ogni sede ha un **token segreto** (store `calendar_tokens`, rigenerabile: la rigenerazione revoca tutte le iscrizioni).
- Endpoint anonimo `GET /api/ics/:token/:feed.ics` (il token è il bearer) con cache 5 min. Feed: `tutti, rilievo, posa, assistenza, altro`.
- ICS RFC 5545 con VTIMEZONE Europe/Rome; titolo = tipo + cliente; location = indirizzo lavoro.
- L'operatore aggiunge il feed in Google Calendar via "Altri calendari → Da URL"; Google lo aggiorna periodicamente (sola lettura lato Google).

38.2 Import (Google → CRM) — overlay in Planning

- Sorgenti per sede (store `external_calendars`): nome, **indirizzo iCal segreto** di Google ("Impostazioni → Integra calendario"), colore da palette, attivo.
- Fetch **server-side** (evita CORS) con cache 10 min, follow redirect, mantiene lo stale su errore; `webcal://` normalizzato a `https`; **SSRF guard** (§3.7).
- Parser ICS interno: unfolding, unescape, DATE/DATE-TIME/UTC, TZID→Europe/Rome; **espansore RRULE** limitato alla finestra richiesta (DAILY/WEEKLY/MONTHLY/YEARLY, INTERVAL, COUNT, UNTIL, BYDAY, EXDATE, cap iterazioni).
- Gli eventi appaiono read-only nel Planning (§12.9).

39. Backup notturno su Google Drive

39.1 Contenuto e struttura

Ogni notte alle **00:00 Europe/Rome** (o manualmente) viene generato uno snapshot datato:

```
- Per documenti e allegati il backup risolve prima `storageKey`, rilegge i byte dal driver attivo e ver
- I record legacy con `dataBase64` restano supportati. Gli allegati ticket sono inclusi anche quando la
Backup CRM YYYY-MM-DD/
  database/<store>.json           ← dump integrale di ogni raccolta (utenti SENZA password)
  Sede <nome>/
    Utenti.json                  ← utenti della sede (sanificati)
    Clienti/<Cognome Nome (CL-id)>/
      Scheda cliente.pdf         ← generata server-side (gemella della scheda in app)
      cliente.json
    Commesse/<CODICE>/
      commessa.json
    Preventivi e contratti/...   ← file caricati, raggruppati per tipo
    Misure/... · Fatture e pagamenti/... · Ordini/... · DDT/... · Foto e altro/...
```

Ticket <id>/... ← allegati ticket
Commesse senza cliente/<CODICE>/...

39.2 Collegamento Google (account personale)

- I service account NON possono scrivere su My Drive personale (Google One) → modalità primaria **OAuth utente**: la direzione collega l'account aziendale (una volta) con scope **drive.file** (l'app vede solo i file che crea).
- Flusso: `backup.oauthStartUrl` (adminProcedure, state monouso anti-CSRF) → authorize Google → callback anonima `/api/oauth/gdrive/callback` → refresh token salvato (store + specchio su file, §28.1).
- Al primo backup l'app crea la cartella **"Backup CRM Ruffino"** nel My Drive collegato; l'operatore può spostarla/condividerla ovunque (tracciata per id — attualmente dentro la cartella condivisa aziendale).
- Fallback: senza credenziali il backup viene comunque scritto su disco server (`backups/` , `gitignored`). Modalità service account mantenuta nel codice per un eventuale passaggio a Workspace/Shared Drive.

39.3 API & UI

- Drive v3 via REST puro (JWT/token con crypto nativo — zero dipendenze npm); find-or-create cartelle con cache per run; upload multipart; `supportsAllDrives` .
- Router `backup` (direzione): `status` , `log` (ultimi 60 run), `runNow` , `updateConfig` , `oauthStartUrl` , `disconnectOAuth` , `checkRoot` .
- Card in Impostazioni: stato ("Drive collegato" + email account), ultimo backup (esito/file/MB), countdown prossimo run, "Esegui ora", collega/scollega, istruzioni una-tantum.
- Single-flight guard (un backup alla volta); log persistito.

40. Fatture in Cloud → Clienti

40.1 Funzione

Ogni 6 ore (se abilitato) o su "Sincronizza ora", il CRM legge le **fatture emesse dell'anno corrente** e **crea i clienti mancanti** (mai tocca le fatture, mai aggiorna clienti esistenti).

40.2 Logica di creazione

- Dedup su denominazione normalizzata contro i clienti esistenti.
- Persone: split cognome/nome **validato col codice fiscale** (consonanti del cognome vs primi 3 caratteri del CF; supporta cognomi composti e denominazioni invertite). CF valido salvato sul cliente.
- Aziende (P.IVA presente o ragione sociale riconosciuta): denominazione completa in `cognome` , tipo `azienda` (o `condominio`).
- I clienti creati vanno sulla sede primaria.

40.3 Config & stato

Store `fic_config`: modalità `oauth` o `manual` , access token e refresh token cifrati, scadenza, `company-Id` , abilitazione, data collegamento e ultimo esito. Lo status non restituisce mai segreti in chiaro.

- OAuth Authorization Code: `oauthStartUrl` crea uno state monouso valido 10 minuti; callback `/api/oauth/fic/callback`; scopes `read-only` `entity.clients:r` `issued_documents.invoices:r`.
- L'access token viene rinnovato prima della scadenza con refresh deduplicato per sede. Il refresh token aggiornato viene persistito cifrato.
- Se l'account espone una sola azienda, questa viene selezionata automaticamente; altrimenti resta il picker `/user/companies`.
- La disconnessione rimuove i token OAuth. Il token manuale resta disponibile soltanto come fallback di emergenza.
- Router direzione: `status`, `oauthStartUrl`, `disconnectOAuth`, `saveConfig`, `companies`, `syncNow`.

Il requisito OAuth è code-complete. L'attivazione in produzione richiede `FIC_OAUTH_CLIENT_ID`, `FIC_OAUTH_CLIENT_SECRET`, `FIC_OAUTH_REDIRECT_URI` e un `MAIL_ENCRYPTION_KEY` stabile su Railway.

41. WhatsApp (deep link)

- Helper `lib/whatsapp.ts`: normalizzazione numeri italiani → `wa.me/39...` con messaggio precompilato; firma aziendale "Ruffino Group — tel. 0187 872687".
- Bottone verde accanto al telefono in: **scheda cliente** (messaggio generico pratica), **scheda commessa** (messaggio con codice commessa), **dialog appuntamento del Planning** (conferma appuntamento con tipo, data e ora).
- Nessun invio automatico: si apre WhatsApp con il testo pronto, l'operatore preme invia.

42. Scheda cliente PDF

- Bottone "Scheda PDF" nella scheda cliente → A4 via jsPDF/autotable: anagrafica completa (tipo, contatti, CF/P.IVA, doppio indirizzo, detrazione, pratica edilizia, assegnatario), note, e tabelle Commesse / Appuntamenti / Ticket / Garanzie con section header e page-break safe.
- La stessa scheda è generata **server-side** per ogni cliente nel backup notturno (§39.1).

43. Migrazione dati 2026 (eseguita il 15/07/2026)

Operazione una-tantum documentata per riferimento storico:

1. **Clienti**: 101 clienti unici estratti dal riepilogo fatture 2026 di Fatture in Cloud (113 righe); 73 creati, 28 già presenti. Split cognome/nome validato via CF; aziende riconosciute da P.IVA.
 2. **Arricchimento**: incrocio con l'export anagrafico completo (846 record) → 72 clienti completati con indirizzo/città/CAP/telefono/email/CF/P.IVA; 1 inversione nome corretta via CF.
 3. **Commesse + stati + note**: dalle 7 liste Microsoft To Do di reparto (Misure Esecutive, Aggiornamento Contratto, Ordini, Produzione, Attesa Posa, Finiture a saldo, Interventi/Regolazioni) → 43 commesse create, 23 riusate, 62 stati portati alla fase corretta, 71 note operative scritte negli step timeline corrispondenti (Firma Contratto / Ordine Merce / Data Spedizione / Appuntamento Posa / Finiture).
 4. **Dedup**: merge di 5 doppioni (typo, nomi invertiti, coniugi cointestatari) con fusione di note timeline e stati; 3 coppie legittime (persona + propria azienda, conviventi) lasciate distinte.
 5. Le righe To Do di clienti non-2026 (fatture 2023-25) sono state escluse deliberatamente.
-

44. Prodotti della commessa

44.1 Scopo

Una commessa deve dire **di cosa si tratta**. Il campo `prodotti[]` esisteva già sul modello ma era riempibile solo dopo, un elemento alla volta, dall'interno della scheda: in lista non compariva nulla.

44.2 Tipologie

Elenco chiuso, per poter raggruppare e filtrare, con "Altro" come valvola di sfogo:

Infissi · Porte interne · Portoncino / Blindato · Zanzariere · Persiane · Avvolgibili / Tapparelle · Cassonetti · Controtelai · Tende da sole · Veneziane · Grate · Vetri · Scale · Altro

`TIPOLOGIE_PRODOTTO` è definito in `server/routers/commesse.ts` e replicato in `client/src/lib/prodotti.ts`: i due elenchi **DEVONO** restare allineati.

44.3 Dichiarazione in creazione

Il dialog **Nuova commessa** — sia nella pagina Commesse sia nella **scheda cliente** — espone un blocco **Prodotti**: righe di tipologia + quantità, aggiungibili e rimuovibili in linea. Le righe finiscono in `prodotti[]` alla creazione. Le righe senza tipologia vengono scartate; la quantità minima è 1.

44.4 Modifica su commesse esistenti

Il tab **Prodotti** della scheda commessa consente aggiunta, modifica ed eliminazione. Il nome del prodotto è la stessa tendina di tipologie; i prodotti già registrati con **nome libero** (precedenti a questa versione) conservano il proprio valore, che viene proposto in cima alla tendina come opzione a sé — correggere una quantità non deve riscrivere in silenzio cos'è il prodotto. Il campo `tipologia` è etichettato **"Materiale"** (PVC / Alluminio / Legno).

Le mutation `addProdotto` / `updateProdotto` / `removeProdotto` **DEVONO** invalidare sia `commesse.by-Id` sia `commesse.list`, altrimenti la colonna in lista resta indietro.

44.5 Presentazione

- **Pagina Commesse**: colonna **Prodotti** con badge `7× Infissi`, `3× Zanzariere`; oltre due voci compaiono `+N` con il dettaglio in tooltip.
- **Scheda cliente**: le card delle commesse mostrano gli stessi badge.

45. Marginalità

45.1 Formula

```
marginale lordo = importoTotale - Σ costi[] - costoPosaStimato
marginale %      = marginale lordo / importoTotale
```

Il calcolo vive in `server/_core/margine.ts` come **funzione pura** (`calcolaMargine`), senza dipendenze dagli store.

Il flag `datiIncompleti` è vero quando manca il pattuito **oppure** non è registrato alcun costo: senza costi il margine risulterebbe un finto 100 %.

45.2 Registro costi (`costi[]`)

I costi si registrano **dentro la commessa**, non dagli ordini fornitore: `{ id, importo, fornitore, descrizione, data, numeroOrdine, note }`. Le mutation `addCosto` / `updateCosto` / `removeCosto` sono riservate a **direzione o amministrazione**.

Scelta deliberata: un solo posto dove scrivere un costo, nessun doppio conteggio. Il modulo Fornitori conserva i propri ordini per la parte logistica (righe, ricevimento merce) ma **non alimenta più il margine** — evitando anche la trappola dell'ordine lasciato in "bozza" che silenziosamente non contava.

`importaCostiDaOrdini` esegue un import **una tantum** degli ordini già a sistema (esclusi `bozza` e `contestato`); è idempotente su `numeroOrdine` e il bottone compare solo finché il registro è vuoto.

45.3 Card "Economia" (scheda commessa)

Visibile **solo** a direzione e amministrazione (la query stessa è gated lato server). Mostra pattuito, costi fornitore con conteggio, costo posa modificabile in linea con blur-save, e margine in € e %.

Colori per fascia: ≥ 30 % verde, **15–30 %** ambra, < 15 % rosso, grigio quando i dati sono incompleti. Sotto, il registro dei costi con modifica in riga ed eliminazione.

45.4 Pagina `/marginalita`

Riservata alla **direzione** (voce di sidebar con flag `direzioneOnly`). Esclude le commesse archiviate.

- **KPI**: margine totale, margine medio %, commesse con dati, dati incompleti.
- **Tabella** ordinabile per % (peggiori prima), margine € o pattuito; ricerca e filtro per stato; le righe con dati incompleti sempre in fondo.
- **Aggregati**: costi per fornitore, margine per venditore, margine per mese di apertura — calcolati solo sulle righe complete.

46. Squadre di posa

46.1 Visibilità

La pagina **Squadre di posa** è in sidebar ed è **leggibile da tutti i ruoli** (`squadre.list` è `protectedProcedure`): serve a chiunque debba sapere chi è in cantiere. Creazione, modifica ed eliminazione restano `adminProcedure`, e la pagina nasconde i comandi a chi non è direzione invece di lasciarlo sbattere contro un FORBIDDEN.

Ogni card squadra elenca le **commesse attive assegnate**, quelle già in fase di posa per prime, ciascuna con link alla commessa.

46.2 Assegnazione alla commessa

Il campo `squadraId` esisteva sul modello ma non era né mostrato né impostabile: le squadre si assegnavano solo al singolo intervento, quindi guardando una commessa non si sapeva chi la stesse posando.

La card **Squadra di posa** nella scheda commessa assegna la squadra con una tendina cercabile. Diventa **ambra con "Da assegnare"** quando la commessa raggiunge `attesa_posa` senza squadra; altrimenti mostra caposquadra e telefono.

Creando un intervento dalla commessa, la squadra della commessa è **pre-selezionata**.

46.3 Board

Le card nelle fasi di posa (`attesa_posa`, `finiture_saldo`, `interventi_regolazioni`) portano un chip con il nome della squadra, oppure **"Squadra da assegnare"** in colore warning quando manca.

47. Storage dei documenti

47.1 Problema

Prima di questa versione ogni documento viveva in base64 dentro la JSONB della propria raccolta (`preventivi_documenti`, `ticket_allegati`). Poiché `persistedStore` riscrive l'intero blob a ogni `save()`, **ogni upload riscriveva tutti i byte di tutti i file**.

47.2 Layer

`server/_core/fileStorage.ts` espone `putFile` / `getFile` / `deleteFileQuiet` con due driver, selezionati da `STORAGE_DRIVER`:

- **local** — file sotto `./data/files/<key>`; adatto allo sviluppo o a un deploy con volume montato;
- **s3** — qualunque endpoint S3-compatibile (Cloudflare R2, AWS S3, MinIO) via REST + **SigV4 firmato con `node:crypto`**, senza dipendenze npm aggiuntive.

Variabili: `S3_ENDPOINT`, `S3_BUCKET`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `S3_REGION`.

47.3 Modello del documento

Il record conserva i soli metadati più `storageKey` e `checksum` (sha256). La lettura è **retro-compatibile**: i record che portano ancora `dataBase64` funzionano immutati, e `byId` ricostruisce il base64 dallo storage così che il client resti identico.

Se il driver fallisce in scrittura, l'upload **ricade sul base64 inline**: un caricamento non deve mai fallire per ragioni infrastrutturali.

47.4 Guardia sul filesystem effimero

Su Railway senza volume il filesystem è effimero: un record otterrebbe `storageKey` e i byte morirebbero al deploy successivo. `putFile` **rifiuta** quando il driver è `local`, l'ambiente è Railway e `STORAGE_ALLOW_EPHEMERAL` non è `1` — e l'upload ricade sull'inline, cioè sul comportamento precedente.

47.5 Migrazione

`scripts/migrate-documents-to-storage.ts` (e le procedure direzione `fileStorage.status` / `fileStorage.migrate`):

- **dry-run di default**, `--apply` per eseguire;
- per ogni documento: scrive → **rilegge** → confronta lo sha256 → **solo allora** rimuove il `dataBase64`;
- **idempotente e riprendibile**: salta chi ha già `storageKey`;
- **rifiuta di partire** senza un backup Drive riuscito nelle ultime 24 h (controlla `backup_log`);
- **rifiuta** il driver `local` su Railway senza opt-in esplicito.

`pnpm storage:check` e la procedura direzione `fileStorage.probe` verificano la configurazione con un ciclo reale `put` → `get` → `checksum` → `delete` senza esporre access key o secret. Il runbook Cloudflare R2 è in `docs/storage-r2.md`.

47.6 Cascade

L'eliminazione di un documento, di un allegato, di un ticket e — nuova — di una **commessa** rimuove anche i byte dallo storage. Prima l'eliminazione di una commessa lasciava i documenti orfani nella raccolta.

47.7 Backup dopo la migrazione

Il backup Drive non legge più soltanto `dataBase64`: usa `storageKey` come fonte canonica, verifica `checksum` e conserva il fallback legacy. Questo requisito è coperto da test per lettura inline, oggetto presente, oggetto mancante e checksum non valido.

48. Formato e parsing degli importi

48.1 Formato

`formatEuro` in `client/src/lib/euro.ts` è l'**unico** formatter: separatore di migliaia col punto, decimali con la virgola, **sempre due cifre decimali** anche quando sono `,00` → `1.234,56`.

`useGrouping: true` è obbligatorio: la regola italiana di `Intl` raggruppa solo da cinque cifre (`minimumGroupingDigits: 2`), per cui `5000` usciva "5000" accanto a "10.000" nella stessa colonna.

Ogni superficie che mostra denaro **DEVE** usare `formatEuro` / `formatEuroSimbolo`: Pagamenti, card acconti, card Economia, Marginalità, feed Dashboard, chip del board, Fornitori, rifacimenti. I preventivatori mantengono il proprio `Intl.NumberFormat` (stampano nei PDF col simbolo in coda) ma con lo stesso flag di raggruppamento.

48.2 Parsing

`parseEuro` interpreta il separatore dal contesto:

- punto e virgola → l'ultimo dei due è il decimale (`1.500,50` = 1500.5);
- solo virgola → decimale (`1500,50`);
- solo punto → **decimale** se seguito da 1–2 cifre (`1500.50`), **migliaia** se seguito da esattamente 3 cifre o se ce n'è più d'uno (`1.500`, `1.234.567`).

Varianti: `parseEuroPositivo` (incassi, > 0) e `parseEuroNonNegativo` (costi, ≥ 0).

Il parser precedente faceva `replace(/\.\/g, "").replace(",", ".")`: corretto per la notazione italiana, **catastrofico** per chi digita il punto decimale — `1500.50` veniva registrato come **150050**, cento volte tanto, senza alcun avviso. Il punto è ciò che quasi tutti digitano sul tastierino numerico. **Nessun `parseFloat` a mano sugli importi.**

49. Correzioni notevoli (v4.2)

Registrate perché ognuna nasconde una regola da non violare di nuovo.

Difetto	Regola che ne deriva
<code>1500.50</code> salvato come <code>150050</code>	il parsing degli importi passa solo da <code>parseEuro*</code> (§48.2)
"Da incassare" calcolato come <code>max(0, Σpattuito - Σincassato)</code> : una commessa incassata in eccesso cancellava il debito di un'altra (4.000 invece di 5.000)	i residui si sommano per commessa , non sugli aggregati; l'eccedenza ha un proprio KPI, un filtro e la dicitura "+€ X in più" sulla riga (prima leggevano "Saldata")
<code>importoIncassato</code> scrivibile via <code>commesse.update</code>	i campi derivati non stanno negli schemi di input
Ticket con commessa cancellata impossibile da eliminare	<code>requireOwnershipOrDirezio(nu ll)</code> lancia <code>NOT_FOUND</code> prima di valutare il ruolo: prevedere sempre il ramo "genitore mancante"
<code>apertoBy</code> sempre <code>null</code> alla creazione	un campo di autorizzazione che non viene mai popolato non autorizza nessuno
Notifiche mute sui ticket senza commessa	ogni sorgente di notifica deve prevedere il caso in cui l'entità collegata manchi
Costo posa che tornava indietro al blur	una scrittura deve invalidare tutte le query che mostrano quel dato, non solo la più ovvia
Colonna Prodotti ferma dopo una modifica dal tab	idem: <code>byId</code> e <code>list</code>
Rata suggerita sempre "1° acconto"	se una <code>list</code> filtra dei campi, ciò che serve alla UI va esposto in forma sintetica (<code>nPagamenti</code>)

50. Tars — agente operativo con approvazione umana

50.1 Principio di sicurezza

Tars **propone, non esegue**. Il modello non possiede strumenti di scrittura diretta sui dati business: crea record in `azioni_suggerite`; un operatore approva o rifiuta; l'esecutore applica la proposta tramite la stessa mutation tRPC usata dall'interfaccia. Pagamenti, avanzamenti di stato, bozze di risposta e collegamenti fattura richiedono ruoli elevati.

Ogni proposta possiede una chiave d'azione canonica derivata da tipo, target e campi significativi del payload. La stessa azione non deve tornare in coda riscritta con parole diverse quando è pendente, approvata, rifiutata, risposta, fallita o già gestita. La similarità del titolo funge da controllo aggiuntivo e il motivo del rifiuto viene restituito al modello.

50.2 Trigger e profili strumenti

Il catalogo tool inviato alla OpenAI Responses API dipende dal trigger:

- `riconciliazione_fatture` : solo FiC, commesse, clienti e pagamenti necessari;
- `smistamento` : comunicazioni, ricerca e collegamento alle entità;
- `gestione_comunicazione` : analisi puntuale di un messaggio con contesto minimo, allegati, collegamento, nuovo lead, ticket e bozza risposta;
- `on_demand` : profilo operativo mirato;
- `audit_processi` : quadro aziendale e strumenti di proposta per miglioramenti misurabili;
- `chat` e `seguito` : catalogo completo quando l'operatore richiede esplorazione.

L'ordine dei tool è stabile per rendere riutilizzabile la cache del provider. Ogni run registra `profiloStrumenti` e `strumentiDisponibili`. Il default è `gpt-5.6-sol` per chat e richieste umane e `gpt-5.6-terra` per i trigger automatici; entrambi sono configurabili per sede.

50.3 Quadro aziendale e confini di accesso

Tars DEVE poter incrociare anagrafiche, commesse, cantiere, economia, comunicazioni, inventario, produzione, qualità e storico delle proprie decisioni. `leggi_quadro_azienza` restituisce una sintesi compatta della sede con KPI, pratiche ferme, scadenze, anomalie e qualità decisionale dell'agente. Gli strumenti verticali permettono di approfondire contenuto documentale, produzione e qualità.

L'accesso ampio non costituisce un bypass: ogni lettura usa il `ctx` applicativo, rimane sede-scoped e rispetta il ruolo. `leggi_organizzazione` e i dati economici richiedono la direzione; credenziali, token e segreti non sono mai restituiti. Il contenuto dei documenti è marcato come fonte esterna non fidata.

50.4 Fascicolo commessa

`leggi_fascicolo_commissa` raccoglie in parallelo dati commessa, timeline, documenti e doc gate, ordini fornitori, magazzino, ticket, interventi e garanzie, restituendo soltanto i campi utili al ragionamento.

Quando `commessaId` è noto all'avvio, il loop precarica il fascicolo nel primo messaggio (`fascicoloPrecaricato=true`), evitando il primo round-trip modello → tool. Una lettura identica successiva è assorbita dalla cache del run.

50.5 Riduzione token e cache

La riduzione token DEVE avvenire senza riusare dati tra utenti o tra esecuzioni:

1. **Prompt caching OpenAI:** prefisso stabile composto da istruzioni, profilo strumenti e cronologia; `prompt_cache_key` deterministico per profilo e modello. Su GPT-5.6 il messaggio developer termina con un breakpoint esplicito e usa modalità `explicit` con TTL 30 minuti; `gpt-5.4-mini` usa il caching implicito. `store=false` evita la conservazione della risposta applicativa; gli item `reasoning.encrypted_content` vengono ripassati nei turni di function calling stateless.

2. **Cache strumenti per run:** ogni `leggi_*` e `cerca_*` usa una chiave JSON stabile; due richieste identiche, anche contemporanee, condividono la stessa Promise. Gli errori non restano in cache.
3. **Profilo minimo:** i trigger automatici non pagano lo schema di strumenti irrilevanti.
4. **Fascicolo compatto:** una lettura aggregata sostituisce numerose chiamate frammentate e output ripetuti.

L'audit salva input, output, cache read, cache write, cache hit strumenti, costo stimato ed esito. Il consumo restituito dal provider viene contabilizzato anche per risposte incomplete o fallite. I campi storici `cache write 5m/1h` restano compatibili con le esecuzioni precedenti; le scritture OpenAI sono contabilizzate nel bucket a moltiplicatore 1,25. La Inbox Tars espone profilo, preload e cache hit per diagnosticare run costosi.

50.6 Audit continuo dei processi

Quando Tars e l'audit sono abilitati, lo scheduler può eseguire una revisione per sede ogni circa 24 ore. Il run automatico usa il profilo minimo `audit_processi`, legge prima il quadro aziendale e DEVE:

- proporre al massimo tre miglioramenti;
- basarsi su pattern ricorrenti o indicatori misurabili, non su un singolo caso;
- descrivere problema, intervento, impatto atteso e metrica di verifica;
- evitare azioni già proposte o decise.

La direzione può avviare l'audit manualmente dalla Inbox Tars e abilitarlo per sede nelle Integrazioni. L'esito resta una proposta: nessuna modifica di processo viene applicata automaticamente.

50.7 Store e budget

Gli store principali sono `azioni_suggerite`, `conoscenza_aziendale`, `agente_esecuzioni`, `agente_config` e `tars_chat`. Il budget mensile e i limiti di esecuzione vengono controllati lato server; l'assenza o il superamento del budget non può degradare in scritture non tracciate.

La configurazione richiede `OPENAI_API_KEY`; `ANTHROPIC_API_KEY` non è più letta. Al raggiungimento di `maxToolCalls`, il loop concede esattamente un turno conclusivo senza strumenti e termina anche se il provider restituisce una risposta anomala: nessun trigger può restare in esecuzione indefinitamente.

51. Comunicazioni (/comunicazioni)

51.1 Modello e ingestione

Email e WhatsApp confluiscono nella tabella `comunicazioni`. La chiave (`casella_id`, `canale`, `message_id`) rende idempotente la sincronizzazione. Oltre a canale, mittente, contenuto, allegati, cliente/com-messa, stato e data, ogni riga persiste categoria, score, motivazione e fonte della classificazione, più l'ulti-mo riepilogo Tars richiesto dall'operatore.

Gli stati sono `nuova`, `vista`, `gestita`. L'eliminazione dal CRM usa `deleted_at`: il messaggio resta nella casella sorgente e il tombstone impedisce che venga importato di nuovo.

51.2 Classificazione e filtro anti-rumore

Le categorie sono `operativa`, `nuovo_lead`, `amministrativa`, `fornitore`, `da_classificare`, `offerta_marketing` e `spam`. `spam` e `offerta_marketing` sono escluse dalla coda e dai conteggi operativi dopo la classificazione, ma restano consultabili nella vista Escluse.

Ogni nuova email in ingresso DEVE passare dalla classificazione automatica di Tars. Il filtro locale usa header del server mail (`X-Spam=*`, `List-Unsubscribe`, `Precedence`), mittente, linguaggio, allegati, regole persistenti e match CRM, ma il suo esito è soltanto una pre-analisi non vincolante. Prima della decisione la comunicazione resta `da_classificare` e visibile.

Tars DEVE chiamare `classifica_comunicazione` una volta per ogni elemento del lotto, registrando categoria, confidenza, presenza di dubbi e motivazione concreta. `spam` e `offerta_marketing` possono essere applicate automaticamente soltanto con confidenza alta e `dubbio=false`; altrimenti il server forza `da_classificare`. Una classificazione manuale dell'operatore non può essere sovrascritta dall'automazione. Se Tars salta un id, la mail resta visibile e viene ritentata dopo un minuto; dopo un errore API il retry avviene al termine della pausa di sicurezza di 15 minuti. In assenza di configurazione o budget la mail non viene nascosta.

Lo scheduler DEVE preservare il risveglio della coda quando un messaggio arriva durante un run o durante una pausa API. Ogni run elabora al massimo 10 messaggi; se il lotto è completo e restano elementi, il successivo parte dopo circa 500 ms. Il primo trigger è debounciato per circa 5 secondi. Un controllo di recupero ogni minuto cerca code residue di tutte le sedi e il bootstrap esegue il primo controllo dopo circa 5 secondi, quindi un deploy o un timer perso non richiedono l'arrivo di una nuova email. Riattivare Tars o modificare il budget risveglia subito la coda.

Una richiesta esplicita di preventivo, sopralluogo, appuntamento o contatto commerciale concreto DEVE essere valutata prima di header spam, segnali newsletter e regole mittente. Se può portare lavoro resta `nuovo_lead`, oppure `operativa` quando il cliente è già riconosciuto, anche se arriva da un indirizzo aziendale o da un portale usato anche per invii massivi. La direzione può memorizzare una regola esatta per mittente; ogni regola è sede-scoped e revocabile, ma non può nascondere una successiva opportunità esplicita.

51.3 Matching e gestione Tars

Il matching deterministico prova riferimenti a commessa/cliente e registra confidenza e motivazione. Tutte le nuove email entrano nello smistamento per essere prima classificate da Tars; dopo la classificazione, solo quelle operative proseguono con proposte di collegamento o gestione.

Dal lettore l'operatore può impartire a Tars un'istruzione sulla singola comunicazione. Il corpo è delimitato come contenuto esterno non fidato e il profilo `gestione_comunicazione` espone soltanto gli strumenti necessari. Se non esiste una commessa, Tars può proporre un ticket senza commessa, una bozza o `proponi_nuovo_lead`.

Per un nuovo lead Tars DEVE prima cercare clienti e commesse esistenti e leggere `leggi_assegnatari`, che restituisce soltanto utenti attivi della sede. Se l'istruzione non indica già una persona in modo inequivocabile, Tars usa `chiedi_chiarimento` e mostra i nomi come opzioni. La risposta riapre una sola volta l'analisi mantenendo il contenuto originale della comunicazione. `proponi_nuovo_lead` rifiuta un assegnatario mancante o non valido.

L'approvazione crea cliente e commessa in stato `preventivo`, imposta lo stesso `assegnatoA` su entrambi e collega la comunicazione tramite le mutation applicative. La card mostra il nome scelto prima dell'approvazione. La chiave canonica usa `comunicazioneId`, quindi lo stesso lead non può essere proposto due volte.

51.4 Inbox operativa

La pagina DEVE offrire:

- code Da gestire, Non collegate, Nuovi lead, Gestite ed Escluse;
- filtro canale Email/WhatsApp, filtro casella e ricerca su mittente, oggetto e testo;
- riga con checkbox, categoria, canale, mittente, ora, oggetto, anteprima, allegati, commessa e stato;
- azioni multiple per chiudere o escludere messaggi con conferma esplicita;
- lettore con stato `In attesa di Tars` o `Dubbio Tars`, fonte, confidenza, motivazione, collegamento commessa con conferma, proposte Tars, allegati e corpo;
- fascia di stato della coda Tars, visibile solo quando serve, con numero di messaggi, anzianità della mail più vecchia, elaborazione/ripresa e cause bloccanti (`disattivato`, chiave mancante, budget esaurito, pausa API);
- area Tars con istruzione libera e preset contestuali per lead, ticket, risposta, aggiornamento commessa e allegati;
- azioni accessibili per gestita/riapri, elimina e ritorno alla lista mobile;
- aggiornamento caselle e gestione configurazione per la direzione.

Su desktop la lista ha larghezza vincolata e il lettore usa `min-width: 0`; su mobile si mostra una vista alla volta. Nessun messaggio, filtro o allegato deve introdurre scroll orizzontale di pagina.

52. Design system, caricamento e analytics

52.1 Identità visuale

Il tema usa Plus Jakarta Sans, fondo grigio caldo, card bianche, testo inchiostro e giallo saturo come accento. Successo, warning, errore e informazione restano cromaticamente distinti. I componenti operativi usano bordi visibili, raggio moderato e ombre leggere; sono vietati colori locali che ricreano una palette fredda o opaca.

52.2 Responsive e tabelle

Clienti, Commesse e Comunicazioni non devono richiedere scroll orizzontale globale. Gli header sticky devono occupare spazio nel flusso e non coprire la prima riga. Le colonne secondarie si nascondono progressivamente; controlli e titoli rifluiscono senza sovrapporsi.

52.3 Code splitting

Ogni pagina è importata con `React.lazy` e caricata dentro un `Suspense` stabile. Il build separa i vendor React, UI, dati e grafici per caching indipendente. Il runtime Manus, JSX location e il debug collector sono ammessi soltanto sul dev server e non devono gonfiare `index.html` di produzione.

52.4 Umami

Lo script Umami viene installato soltanto in produzione, con endpoint HTTP(S) valido e website id presenti. Ha un id univoco per evitare duplicati, usa `async / defer` e si rimuove in caso di errore di caricamento. In sviluppo non deve generare richieste o warning console.